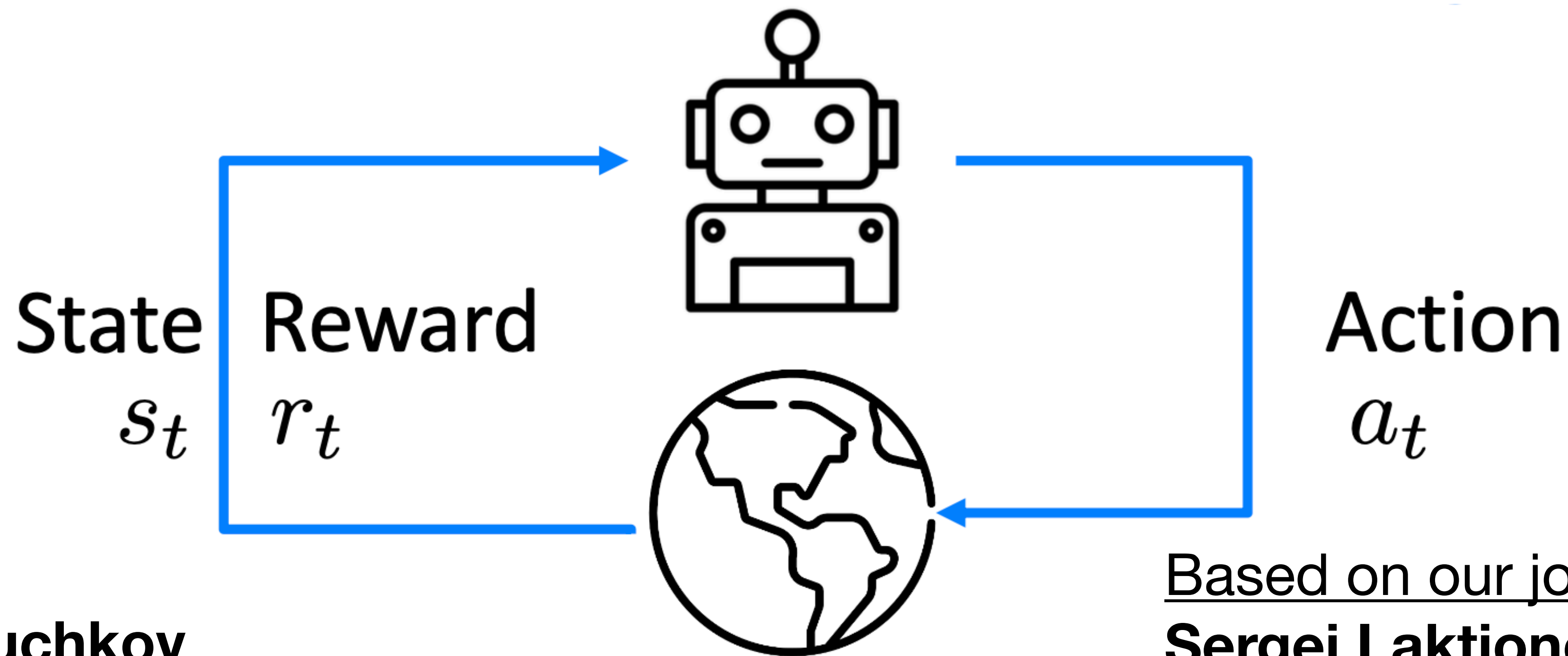


Introduction to Reinforcement Learning

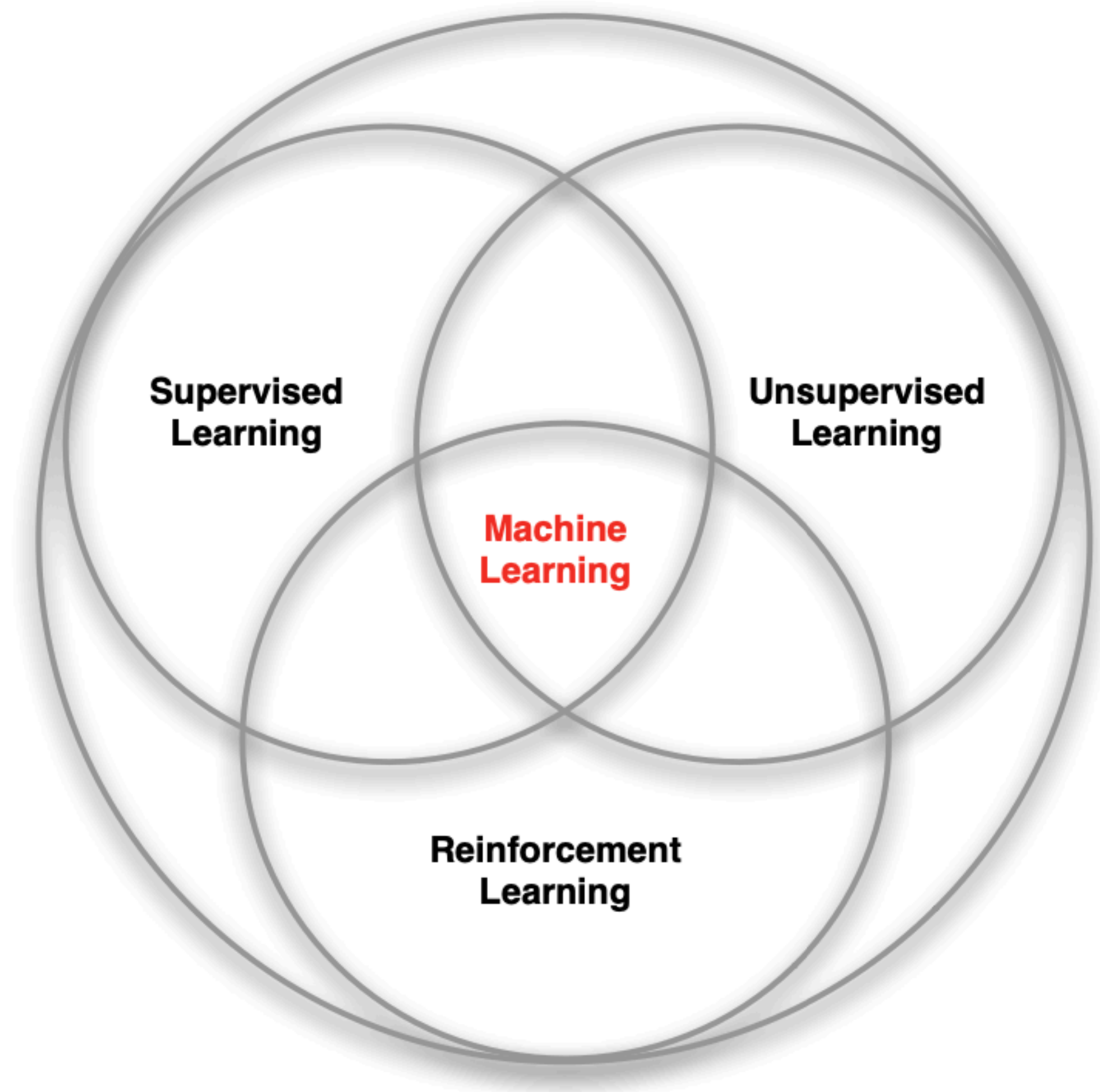
When to apply RL vs supervised learning?



Viacheslav Buchkov
[LinkedIn](#)

Based on our joint course with:
Sergei Laktionov
[LinkedIn](#)

Recap



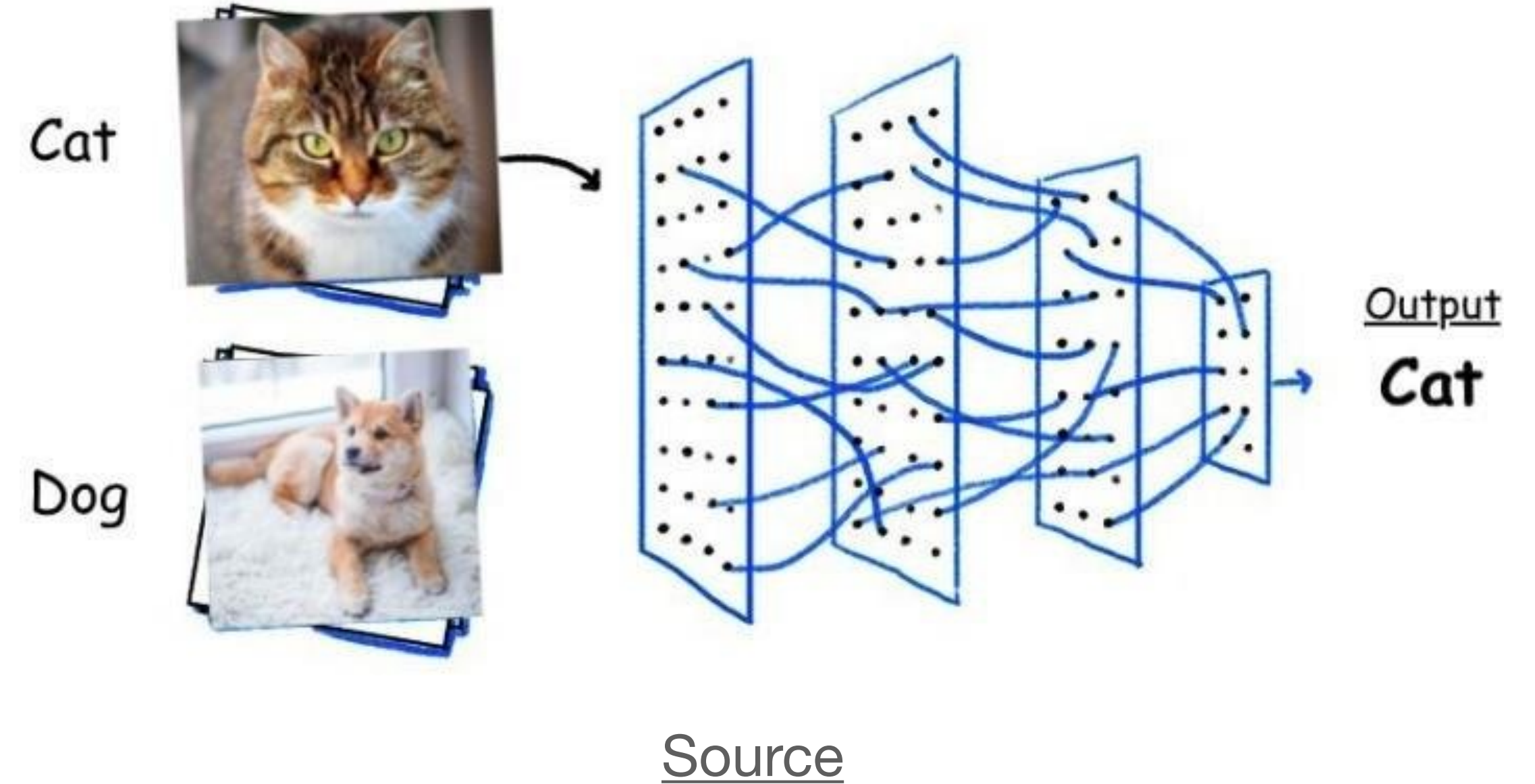
Source

Supervised Learning

Train sample: $(x_i, y_i)_{i=1}^n \sim i.i.d.$

Approximation: $y_i \approx \hat{y}_i = f(x_i)$, where $f \in \mathcal{F}$, $\mathcal{F}$ is a family of algorithms

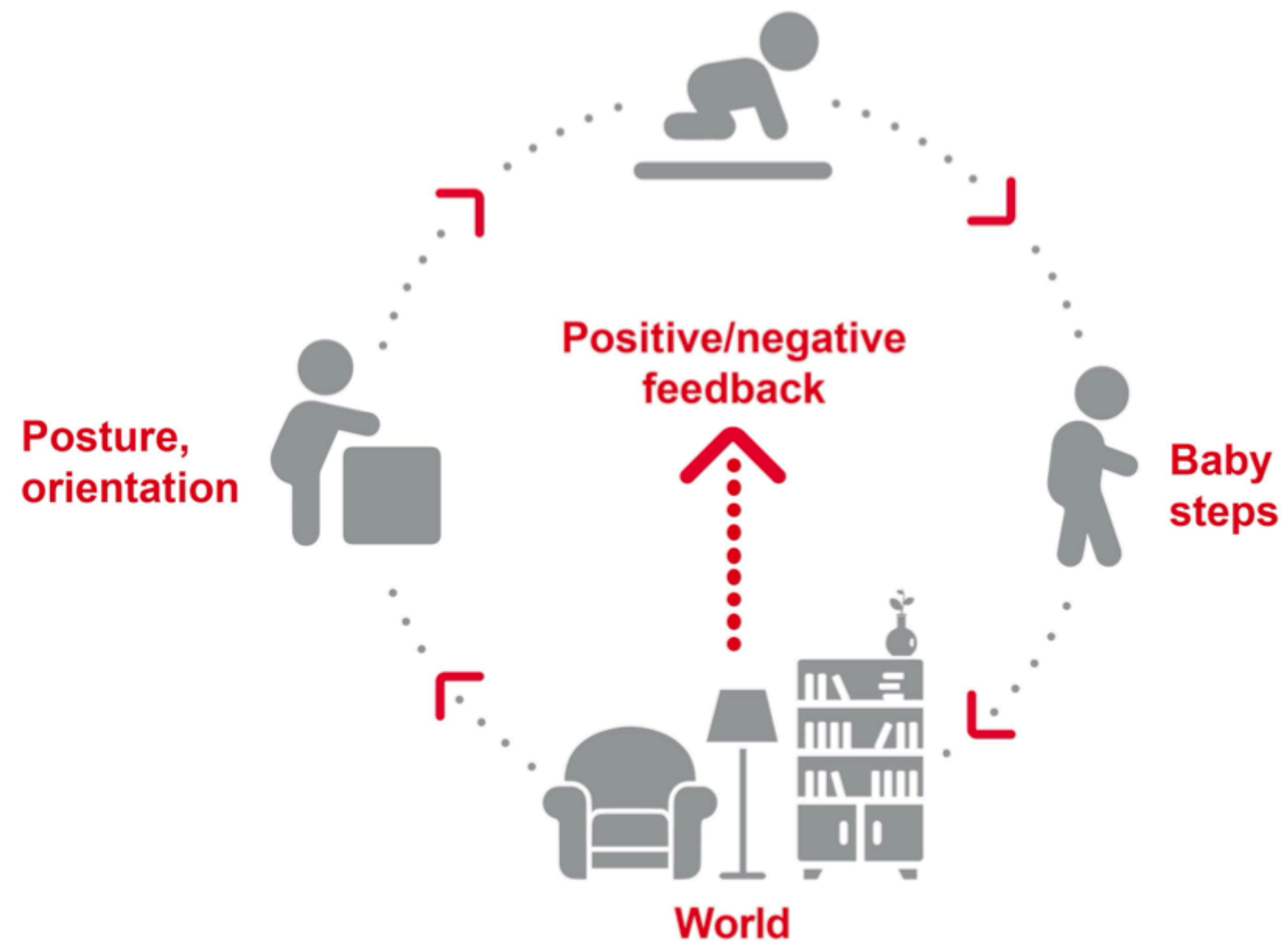
Loss minimisation: $\frac{1}{n} \sum_{i=1}^n L(f(x_i), y_i) \rightarrow \min_{f \in \mathcal{F}}$



Sequential Decision Making

Learning to act by **sequentially** *interacting* with an **uncertain environment**

Baby learning



Source

Self-driving car



Source

Sequential Decision Making

Learning to act by **sequentially** *interacting* with an **uncertain environment**

World: “You are in state 17. You can take actions 3 and 9”

Agent: “I take 3 ”

World: “You get reward -4 and are now in state 279. You can take actions 7 and 9 ”

Agent: “I take 9 ”

World: “You get reward 27 and are now in state 281 ... You can take actions 2 and 17”

...

Assumption: States change according to some (unknown) MDP!

RL vs Supervised Learning

Reinforcement Learning:

- **No supervisor**, only reward signal, obtained by interaction
- Need to **collect samples ourselves**
- Potentially delayed feedback
- Agent's actions affect future samples that it receives
- Exploration-exploitation trade-off

Supervised Learning:

- Correctly **labelled examples**
- **Pre-defined** set of data
- Can **instantly assess** correctness
- Our predictions do not affect future samples
- Data is collected independently of the model's output

Takeaway: RL is essential in the case of learning to act by sequentially interacting with an uncertain environment

RL vs Supervised Learning

Reinforcement Learning:

- **No supervisor**, only reward signal, obtained by interaction
- Need to **collect samples ourselves**
- Potentially delayed feedback
- Agent's actions affect future samples that it receives
- Exploration-exploitation trade-off

Supervised Learning:

- Correctly **labelled examples**
- **Pre-defined** set of data
- Can **instantly assess** correctness
- Our predictions do not affect future samples
- Data is collected independently of the model's output

Caveat → “Virtue of RL Generality”

RL vs MPC

Reinforcement Learning:

- Aims to learn in **unknown** MDP setting
- Learn policy and use it outright (cheap forward pass)
- Doesn't naturally encode any safety constraints (→ area of research)

Model Predictive Control:

- Requires explicit (or very low-error) system dynamics
- Each time solve a finite-horizon problem for a new decision
- Can explicitly encode safety constraints

Takeaway: RL is more general approach that comes at the cost of sample efficiency

RL vs EA

Reinforcement Learning:

- Exploits MDP structure to converge faster
- Requires *some* regularity (see later in the lectures)
- Can reuse samples, collected by other policies / experts

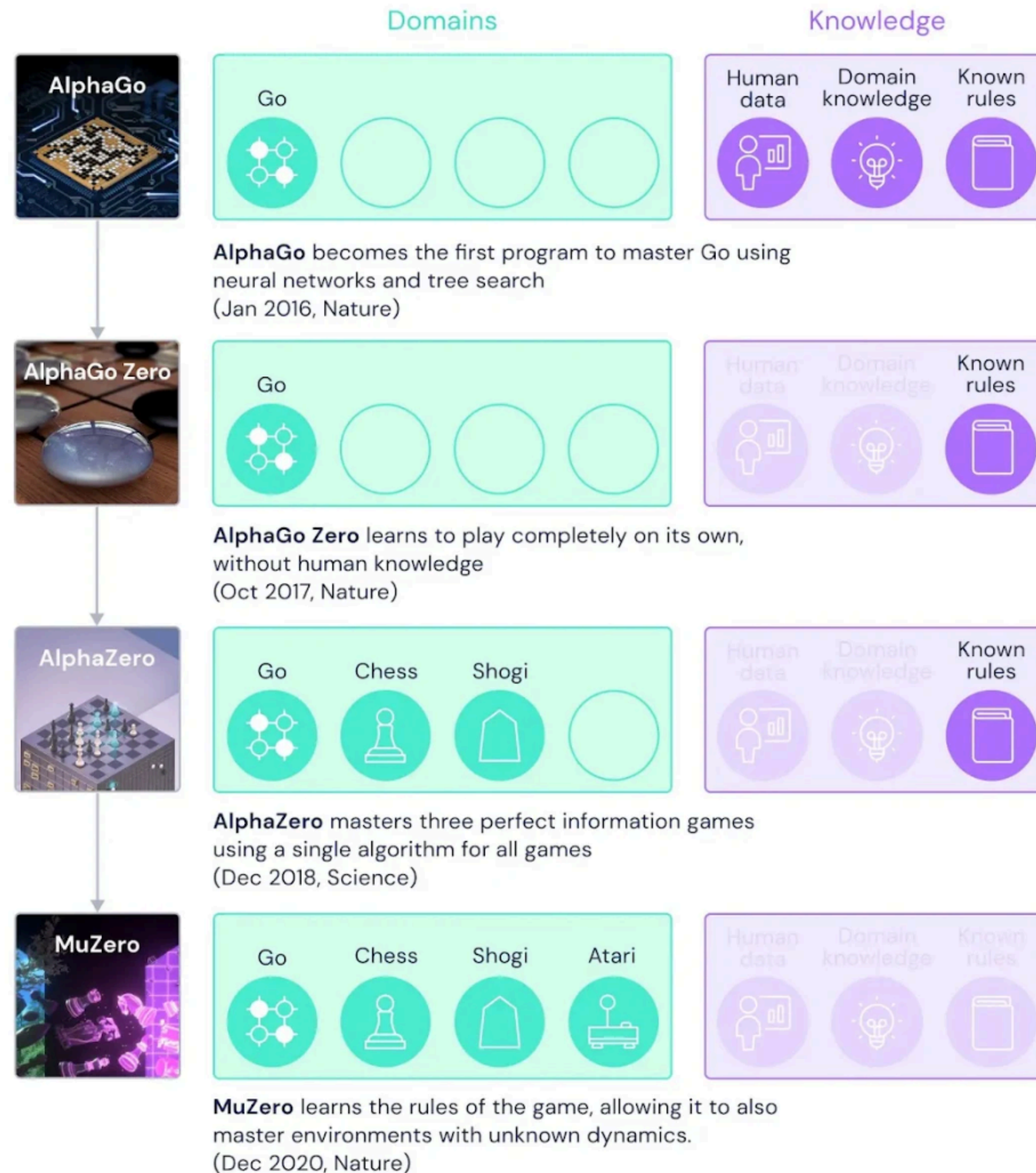
Evolutionary Algorithms:

- **Very sample inefficient**
- Works with non-differentiable, discontinuous, or noisy objectives
- Generally cannot implement in *off-policy* (see later) manner

Takeaway: RL is more sample efficient way to encode minimal assumptions about the environment

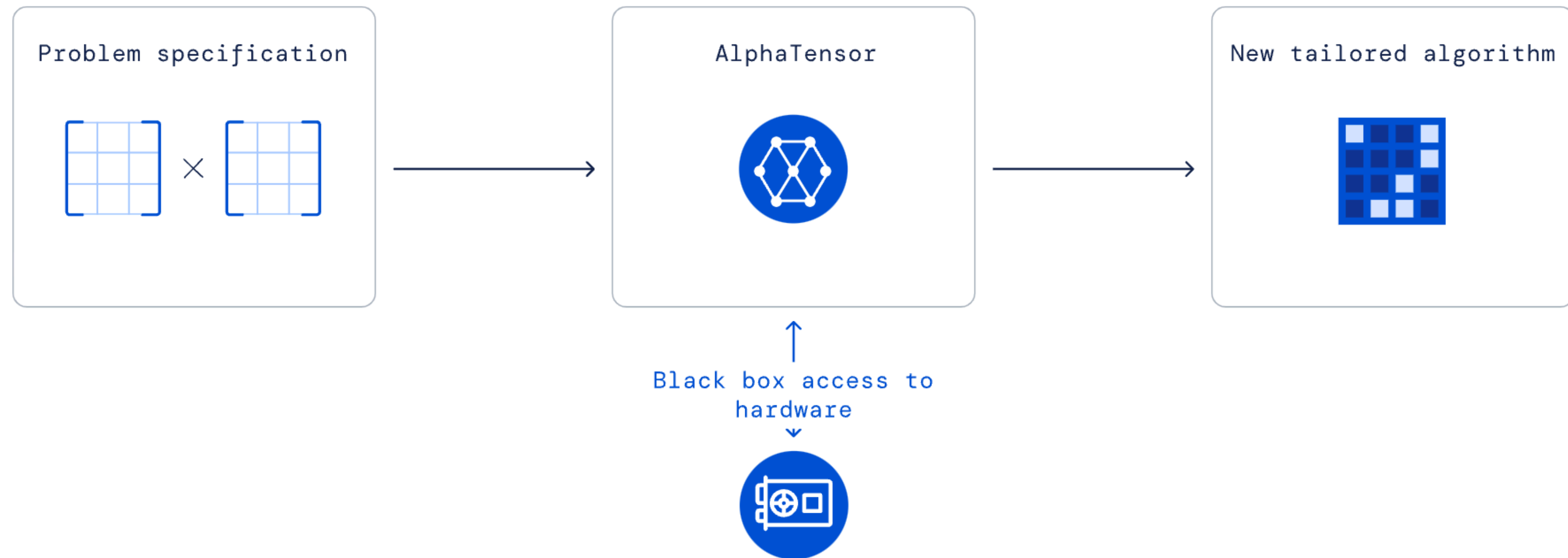
Examples: Board Games

<https://deepmind.google/discover/blog/muzero-mastering-go-chess-shogi-and-atari-without-rules/>



Examples: Discovering Novel Algorithms

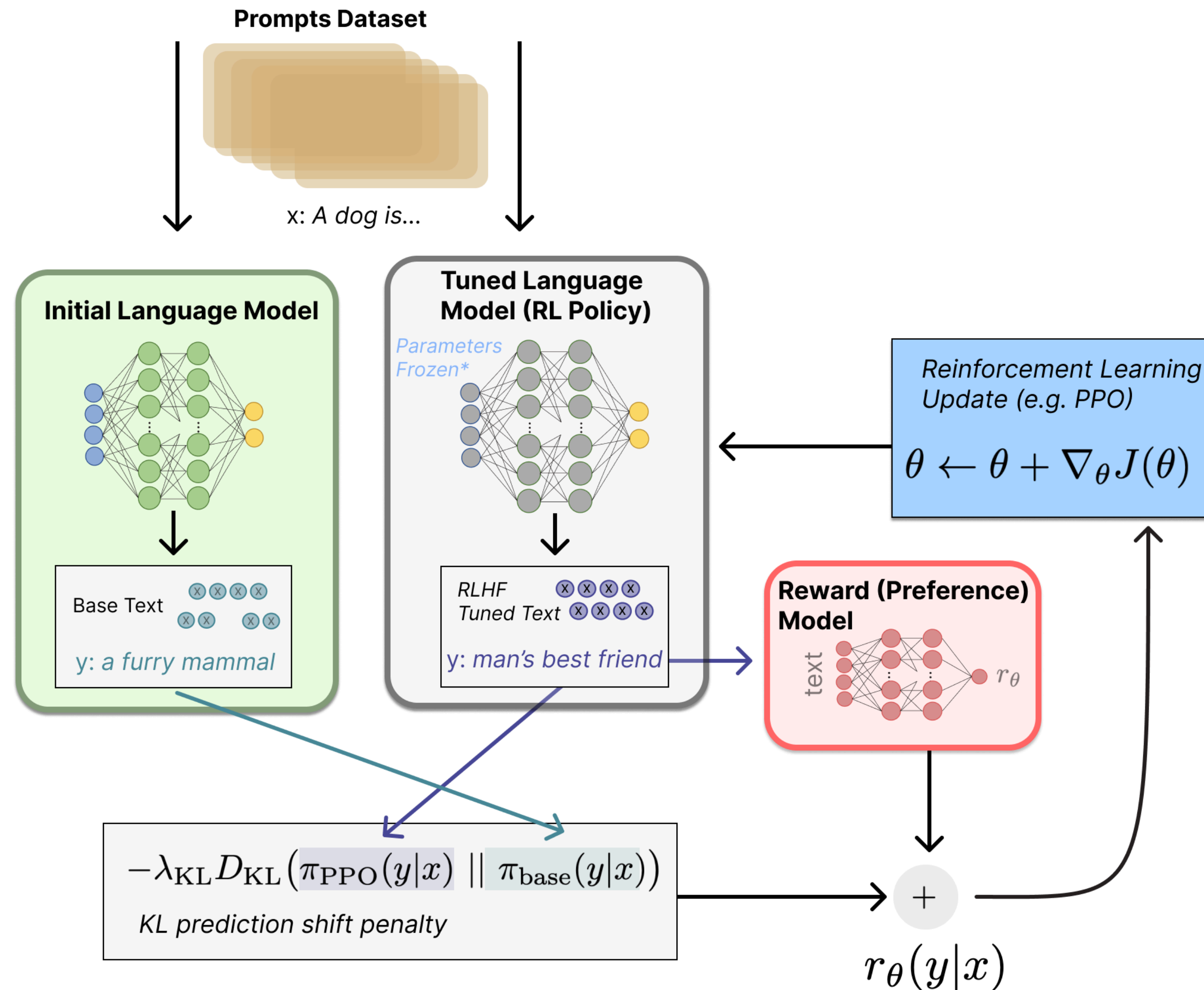
<https://deepmind.google/discover/blog/discovering-novel-algorithms-with-alphatensor/>



AlphaTensor with an objective corresponding to the runtime of the algorithm. When a correct matrix multiplication algorithm is discovered, it's benchmarked on the target hardware, which is then fed back to AlphaTensor, in order to learn more efficient algorithms on the target hardware.

Examples: LLMs Alignment

<https://huggingface.co/blog/rlhf>



Examples: Trading

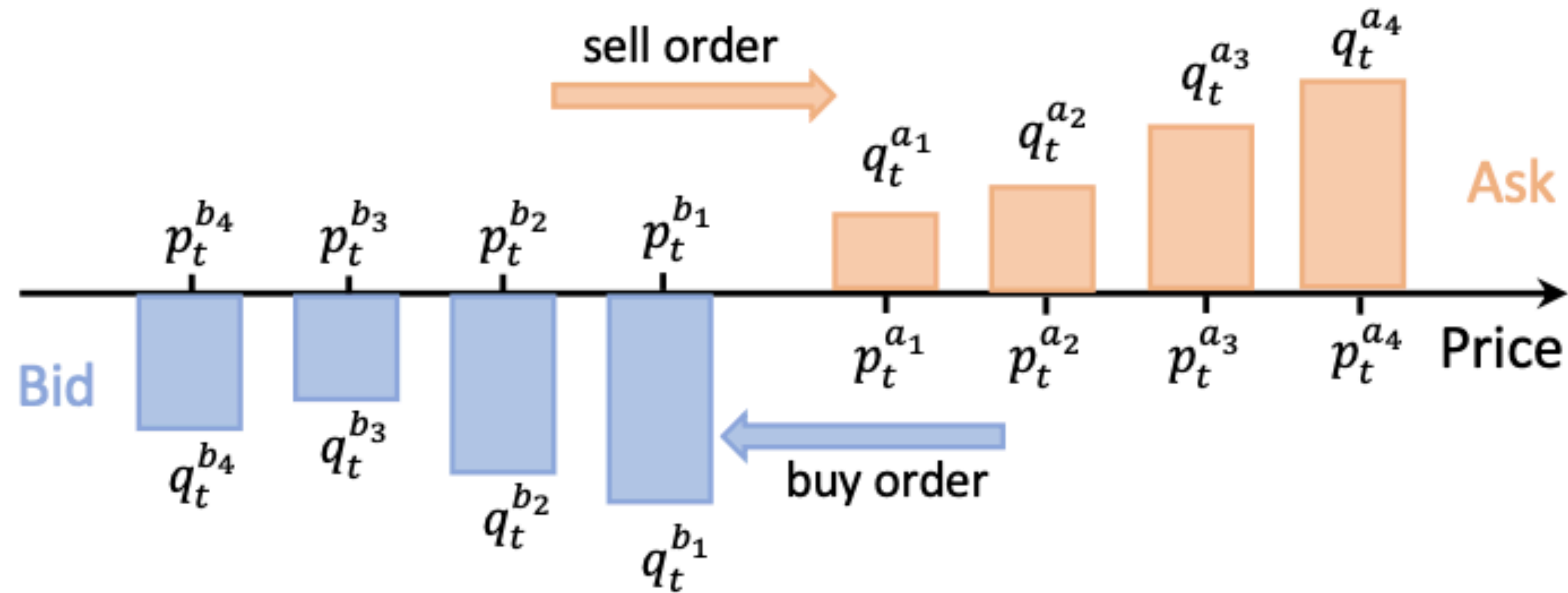


Fig. 3. A Snapshot of Limit Order Book.

Source

Another Example

RL Taxonomy

RL in a Nutshell

Where an ML Researcher can see such a task?

- **Active Learning**

- What if the samples for supervised task are not provided and are expensive to collect?

- **Bayesian Optimization**

- What if we want to optimize a function (supervisor unavailable) only by interacting with it?

- **Tabular RL / Deep RL**

- What if our actions affect the data that we collect, making it not an i.i.d., but a sequential task?

- **Inverse RL**

- What if we cannot interact with the environment, but only have expert demonstrations?

RL in a Nutshell

Where an ML Researcher can see such a task?

- **Active Learning**

- [*Memory-Limited NN*](#), [*Medical Imaging*](#), [*Test-Time LLM Fine-Tuning*](#)

- **Bayesian Optimization**

- [*Advertisement*](#), [*COVID-19 testing*](#), [*Optuna Hyperparam Tuning*](#), [*Financial Portfolio Optimization*](#)

- **Tabular RL / Deep RL**

- [*LLM Tuning*](#), [*Robotics*](#), [*Ride-Sharing*](#), [*Video Games*](#), [*RecSys*](#), [*Limit Order Execution*](#)

- **Inverse RL**

- [*Simulated highway driving*](#), [*Autonomous driving*](#), [*Helicopter Acrobatics*](#), [*Human goal inference*](#)

Setup



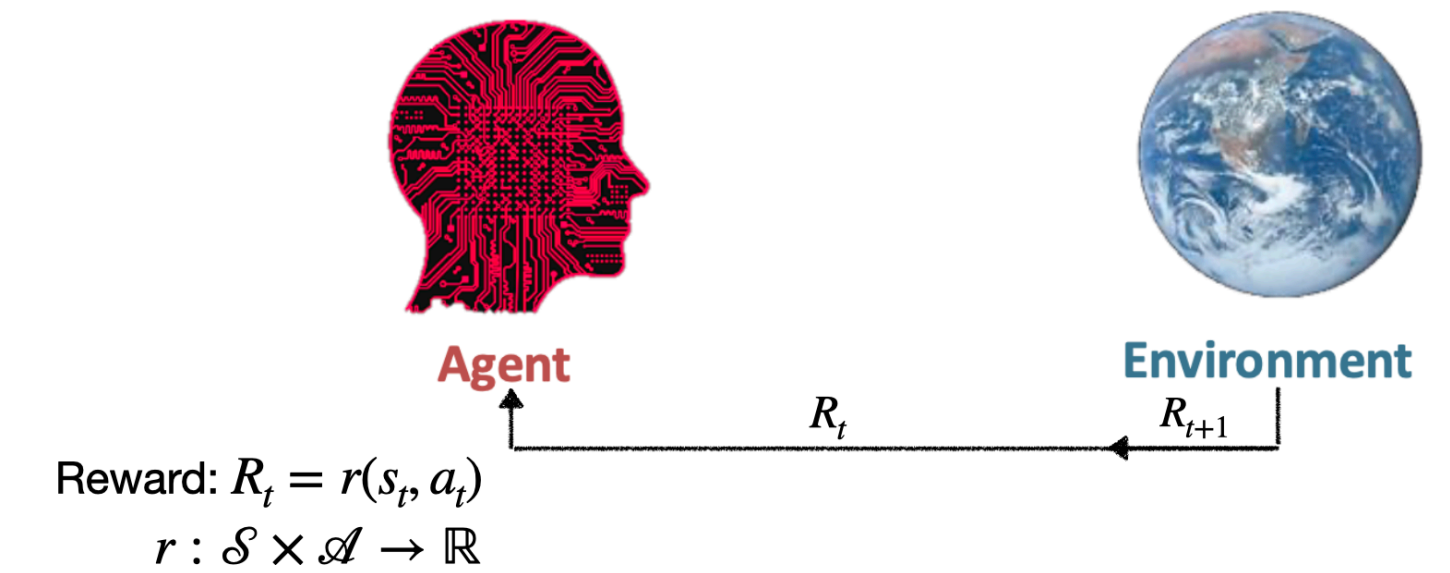
Agent



Environment

Reward

- A reward $R_t \in \mathbb{R}$ is a **scalar** feedback signal
- Indicates how well agent is doing at step t
- The agent's job is to maximise cumulative reward
- Can be deterministic $R_t = r(s_t, a_t)$ or stochastic $R_t \sim r(s_t, a_t)$
- Can depend on new state too $r(s_t, a_t, s_{t+1})$

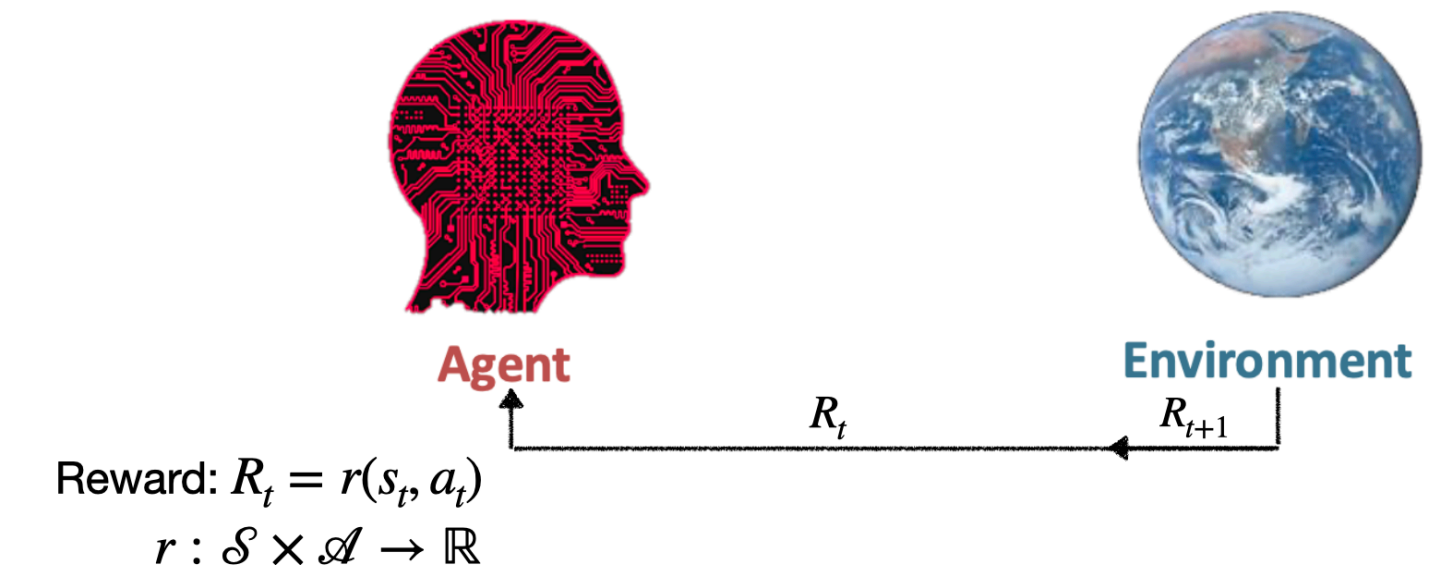


Reward hypothesis (Richard Sutton):

All goals can be described by the
maximisation of expected cumulative reward

Reward

- A reward $R_t \in \mathbb{R}$ is a **scalar** feedback signal
- Indicates how well agent is doing at step t
- The agent's job is to maximise cumulative reward
- Can be deterministic $R_t = r(s_t, a_t)$ or stochastic $R_t \sim r(s_t, a_t)$
- Can depend on new state too $r(s_t, a_t, s_{t+1})$



Reward hypothesis (Richard Sutton):

All goals can be described by the
maximisation of expected cumulative reward

Caveat → Can be delayed / sparse / deceptive → *Credit assignment problem*

Reward Design

... is a tricky task



[Link](#)

Reward Design

Sparse rewards example:

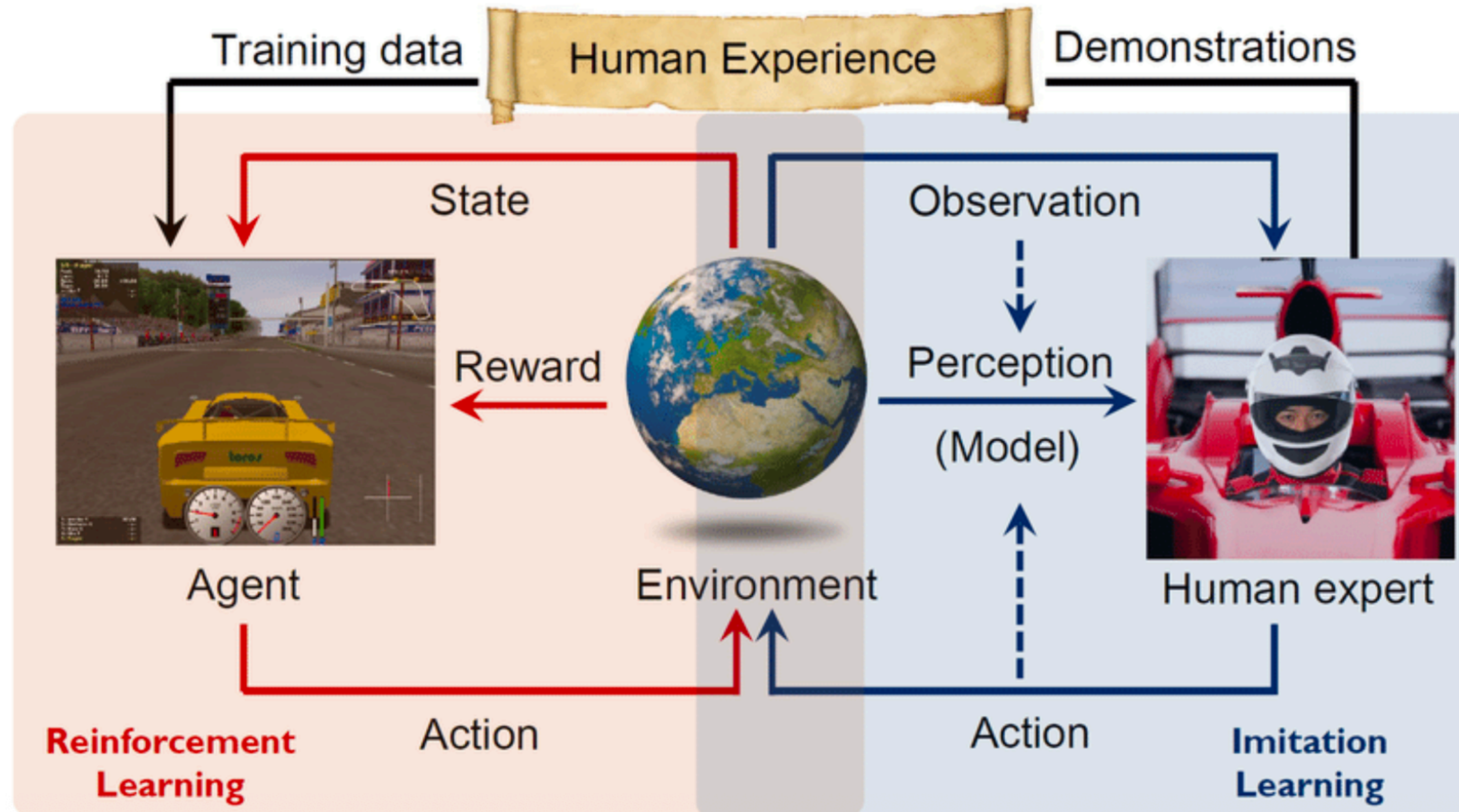


$$r(s, a) = \begin{cases} 1, & \text{if } \|s - s^{Goal}\|_2 \leq \epsilon \\ 0, & \text{else} \end{cases}$$

Illustration was generated with Gemini

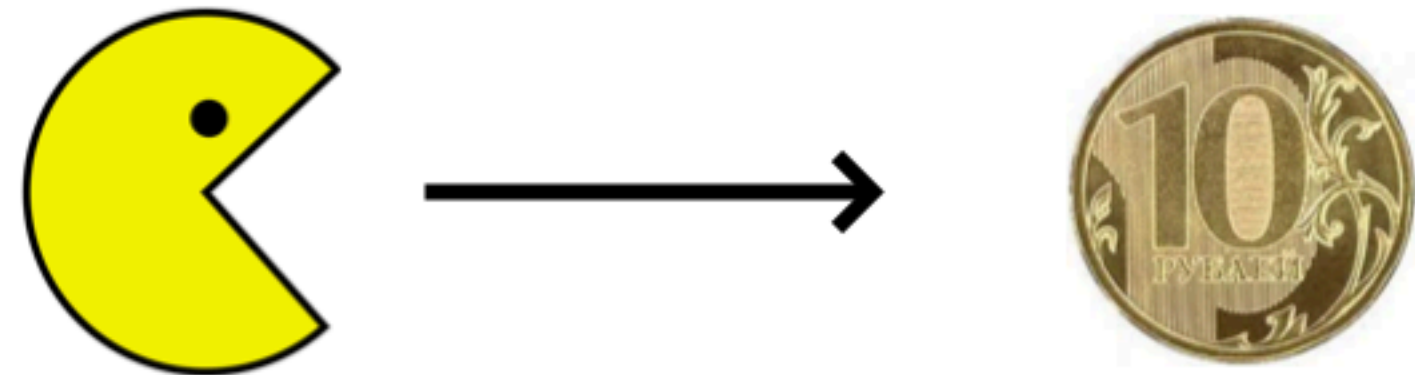
Reward Design

What if we do not know how to design a reward?

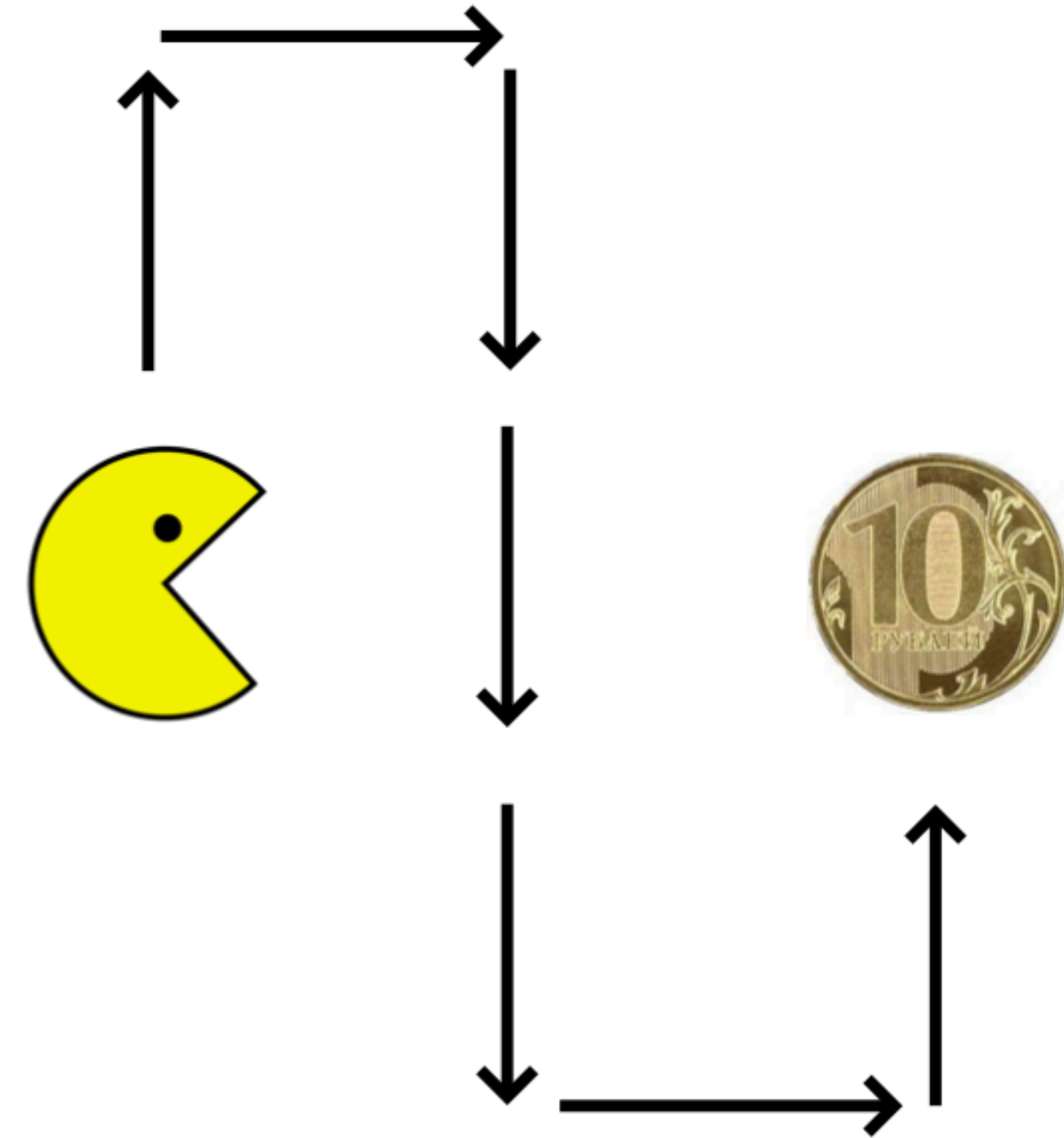


Source: Zuo et al., 2017

Discount Factor



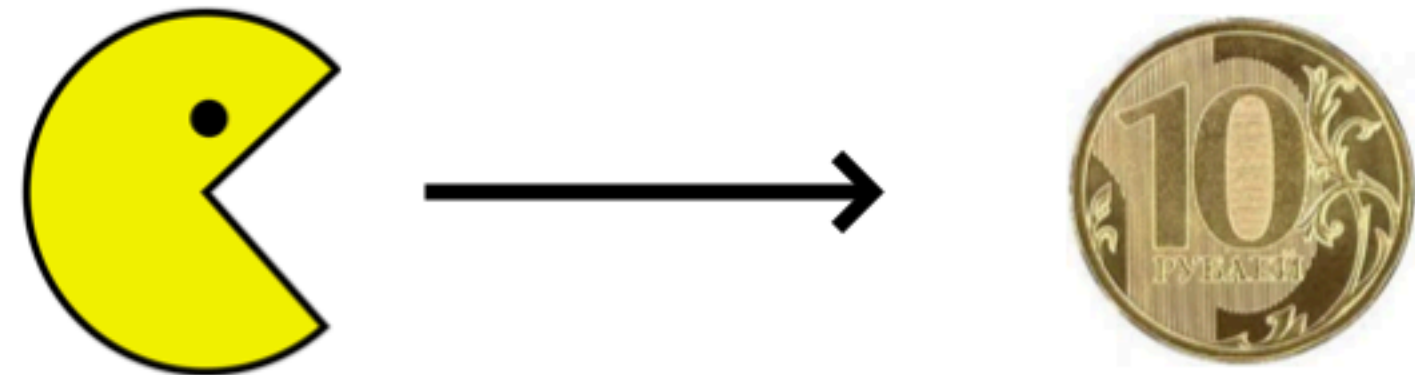
Source



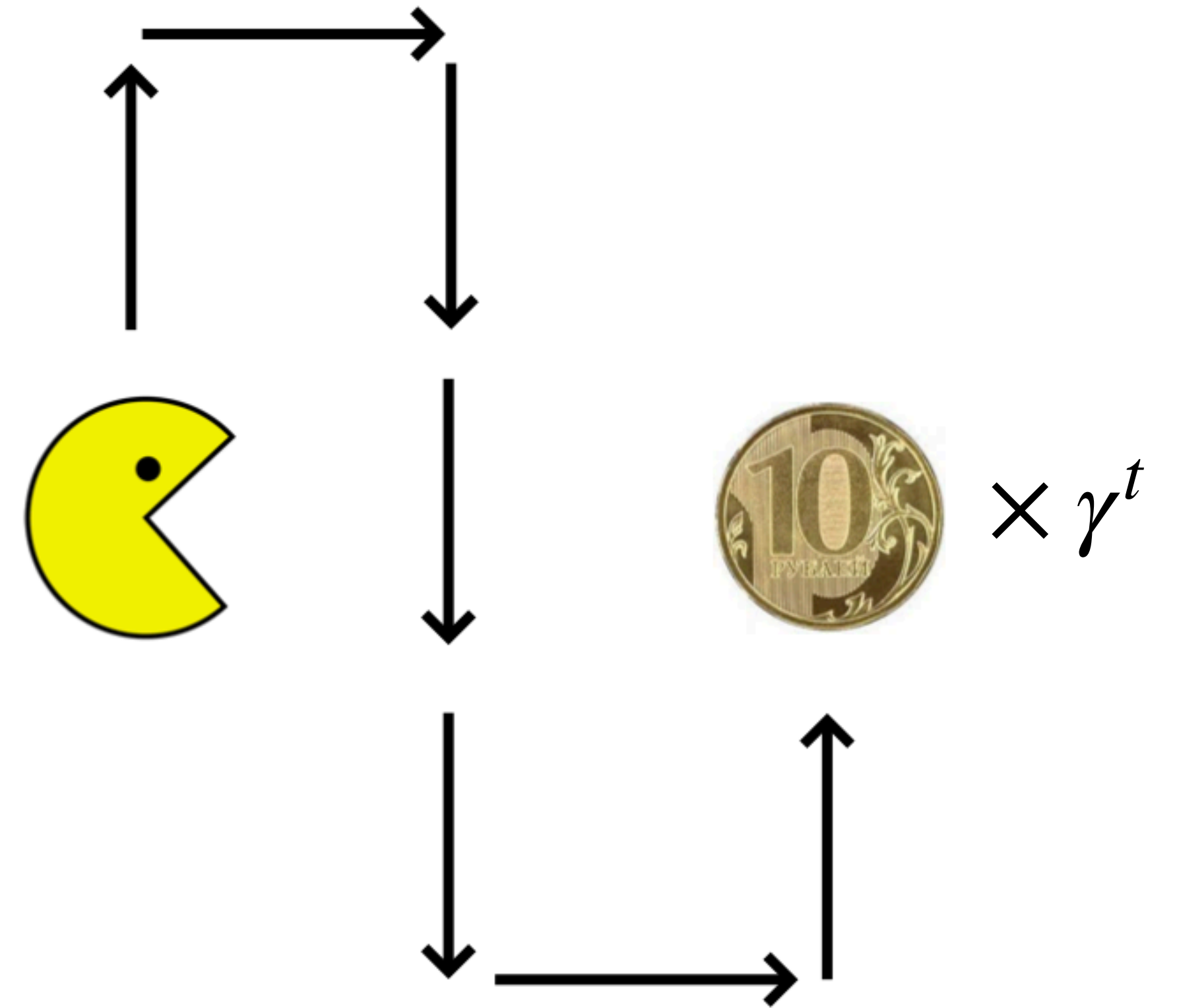
Source

Discount Factor

$\gamma \in [0,1]$ is a discount factor



Source



Source

Why Discount?

$\gamma \in [0,1]$ is a discount factor

- Avoids infinite returns: $\sum_{t=0}^T r_t$ or $\sum_{t=0}^{\infty} \gamma^t r_t$
- Uncertainty about the future may not be fully represented;
- Immediate rewards may earn more interest than delayed rewards due to volatility;
- Animal/human behaviour shows preference for immediate reward;
- It is sometimes possible to use undiscounted reward, e.g. if all sequences terminate;

Why Discount?

$\gamma \in [0,1]$ is a discount factor

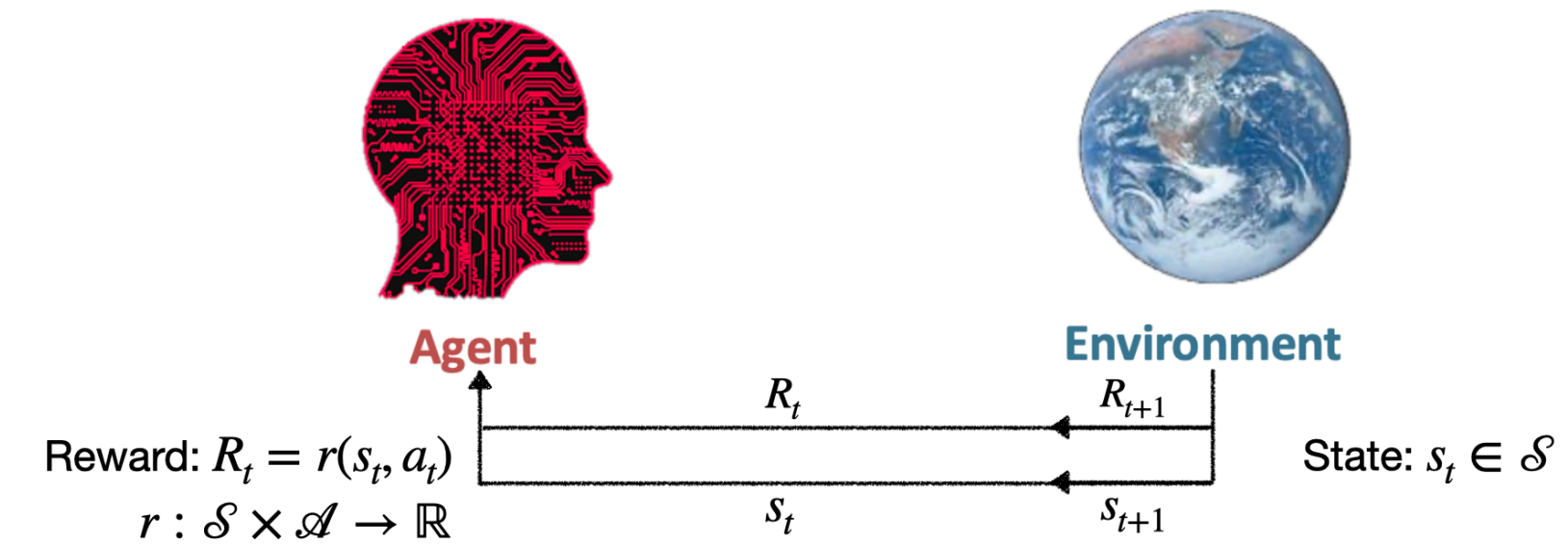
- Avoids infinite returns: $\sum_{t=0}^T r_t$ or $\sum_{t=0}^{\infty} \gamma^t r_t$
- Uncertainty about the future may not be fully represented;
- Immediate rewards may earn more interest than delayed rewards due to volatility;
- Animal/human behaviour shows preference for immediate reward;
- It is sometimes possible to use undiscounted reward, e.g. if all sequences terminate;
- γ controls “effective horizon”: for any $\epsilon > 0$, $T \geq \tilde{\mathcal{O}}(\frac{1}{1-\gamma} \log \frac{D}{\epsilon}) \implies G_t \leq \epsilon$

Why Discount?

$\gamma \in [0,1]$ is a discount factor

- Avoids infinite returns: $\sum_{t=0}^T r_t$ or $\sum_{t=0}^{\infty} \gamma^t r_t$
- Alternatively, one may view discounting as **adding some terminal state**:
 - We have **infinite horizon problem** with state space $S = \{s_1, s_2, \dots, s_{|\mathcal{S}|}\}$
 - Define an auxiliary problem with a virtual terminal state $\tilde{S} = S \cup \{s_0\}$
 - With probability $1 - \gamma$ we randomly transition to this terminal state, otherwise we continue: $\tilde{P}_{ij}(a) = \gamma P_{ij}(a)$ and $\tilde{P}_{i0}(a) = 1 - \gamma$
 - We can show that **there is one-to-one mapping of any policy π in the discounted setting and the auxiliary problem $\tilde{\pi}$**

State



- Defines the law of systems' behavior at the given timestamp $s_t \in \mathcal{S}$
- State space be discrete $\mathcal{S} = \{s_1, s_2, \dots, s_n\}$

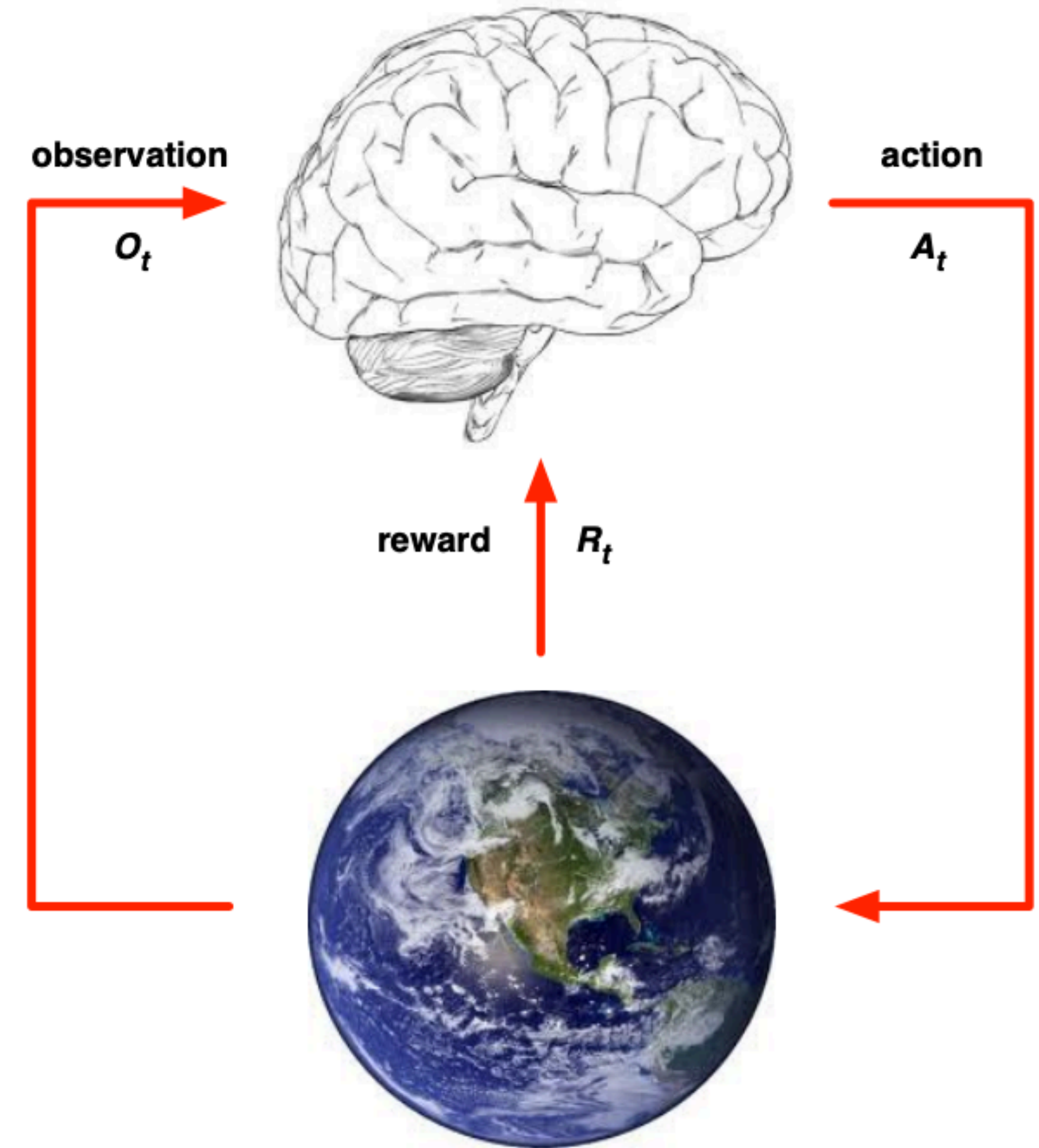
Example: $\mathcal{S} = \{WW, WL, LW, LL\}$

- State space be continuous $\mathcal{S} \subseteq \mathbb{R}^d$

Example: $s_t = [\text{flattened } 64 \times 64 \text{ picture}]$

Agent and Environment

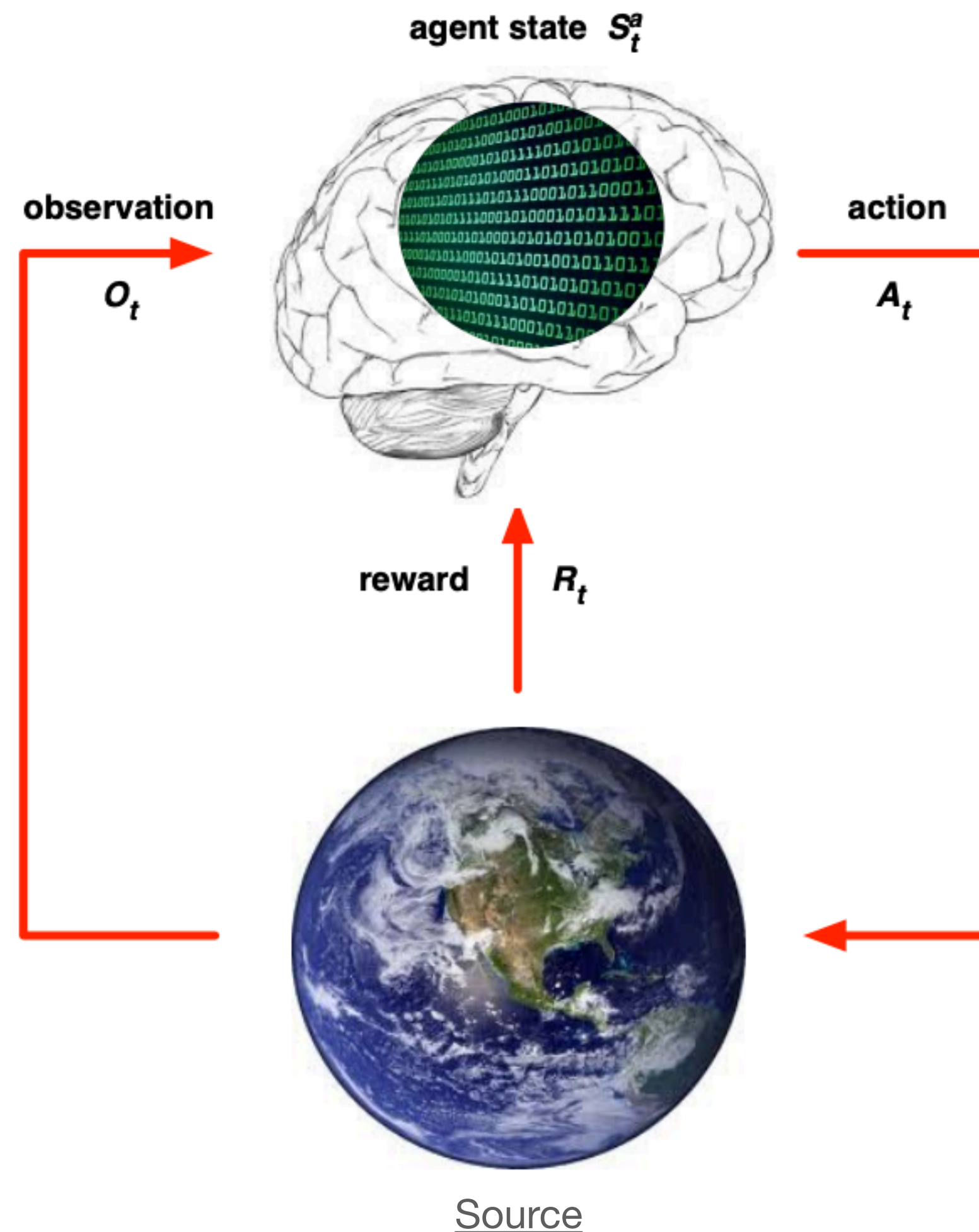
- At each step t the agent being at O_t :
 - Executes action A_t
 - Receives scalar reward R_t
 - Receives next observation O_{t+1}
- At each step t the environment:
 - Receives action A_t
 - Emits reward R_t
 - Emits observation O_{t+1}



Source

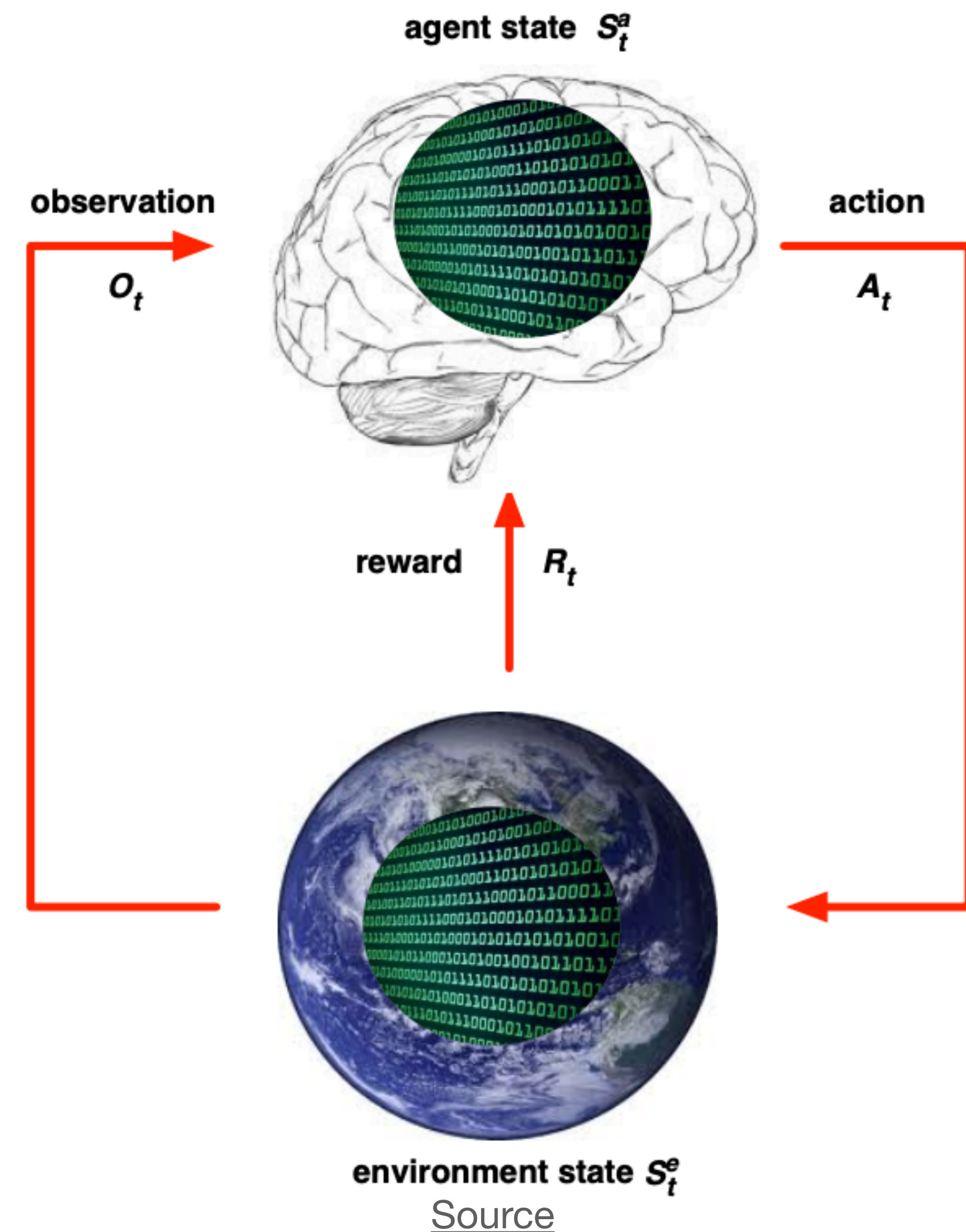
States

The environment state S_t^e is the environment's private representation.
The agent state S_t^a is the agent's internal representation.

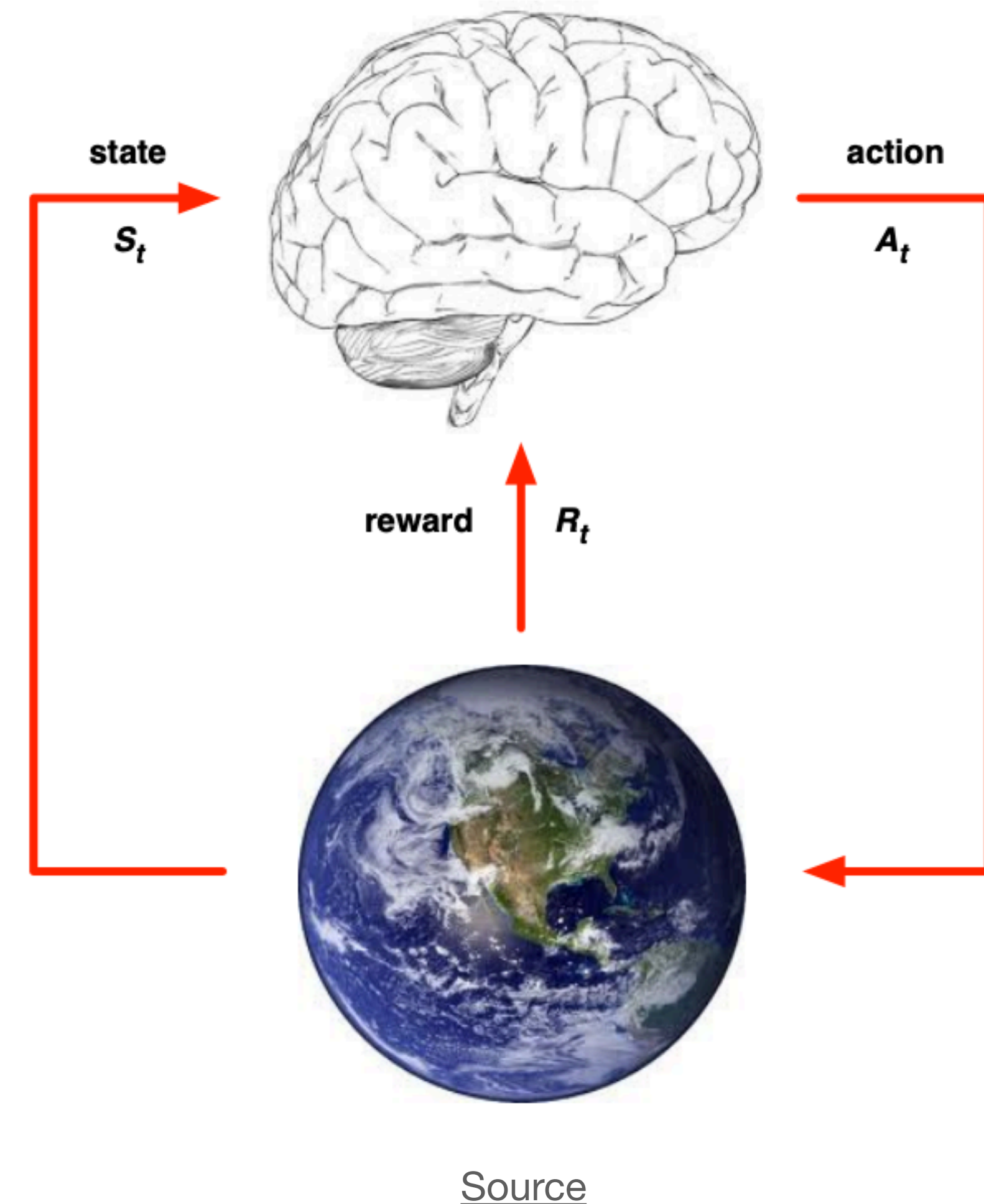


Observability

Partially observable: $S_t^e \neq S_t^a$



Fully observable: $O_t = S_t^e = S_t^a$



Objective

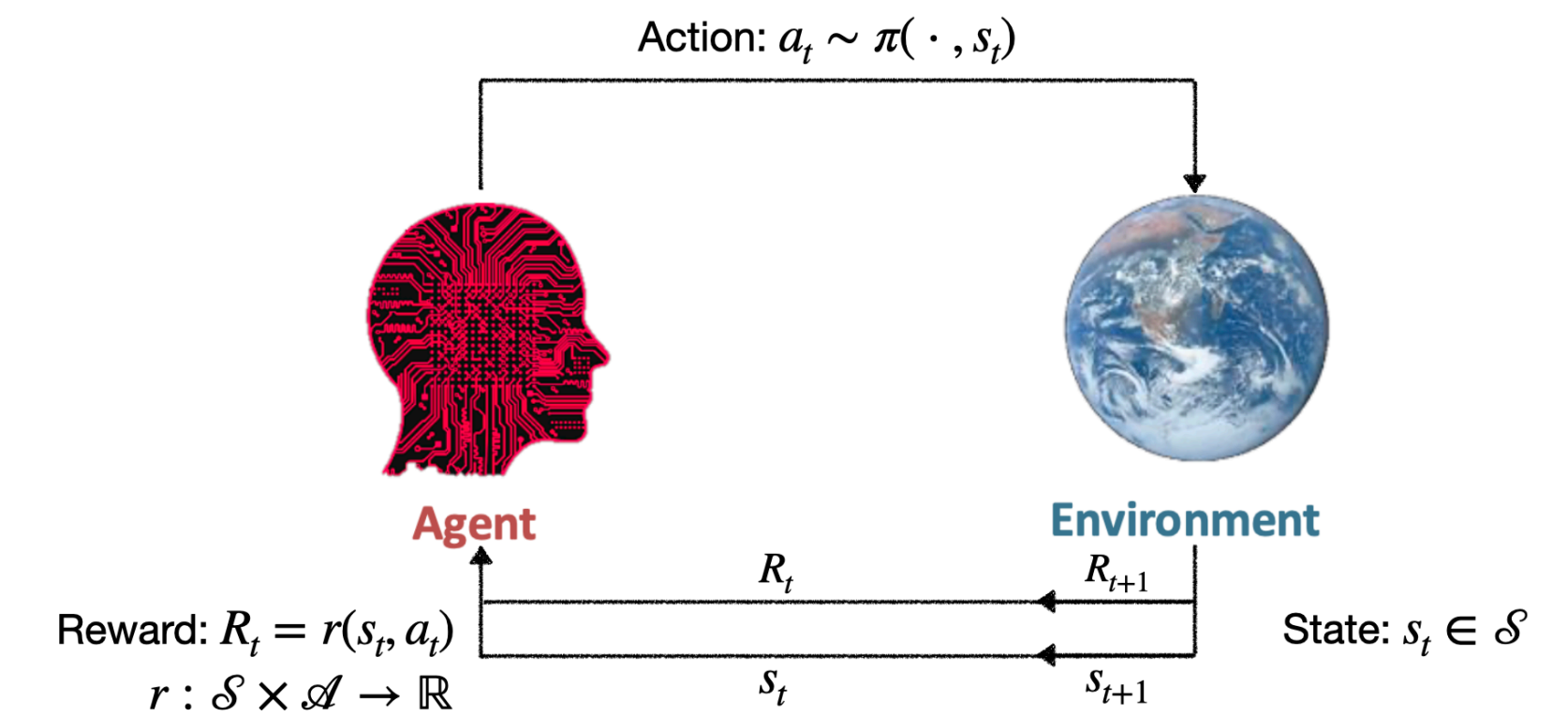
Goal: maximize expected cumulative reward

$$G_0 = \sum_{t=0}^T \gamma^t R_t$$

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t R_t \right] \rightarrow \max_{\pi}$$

Policy

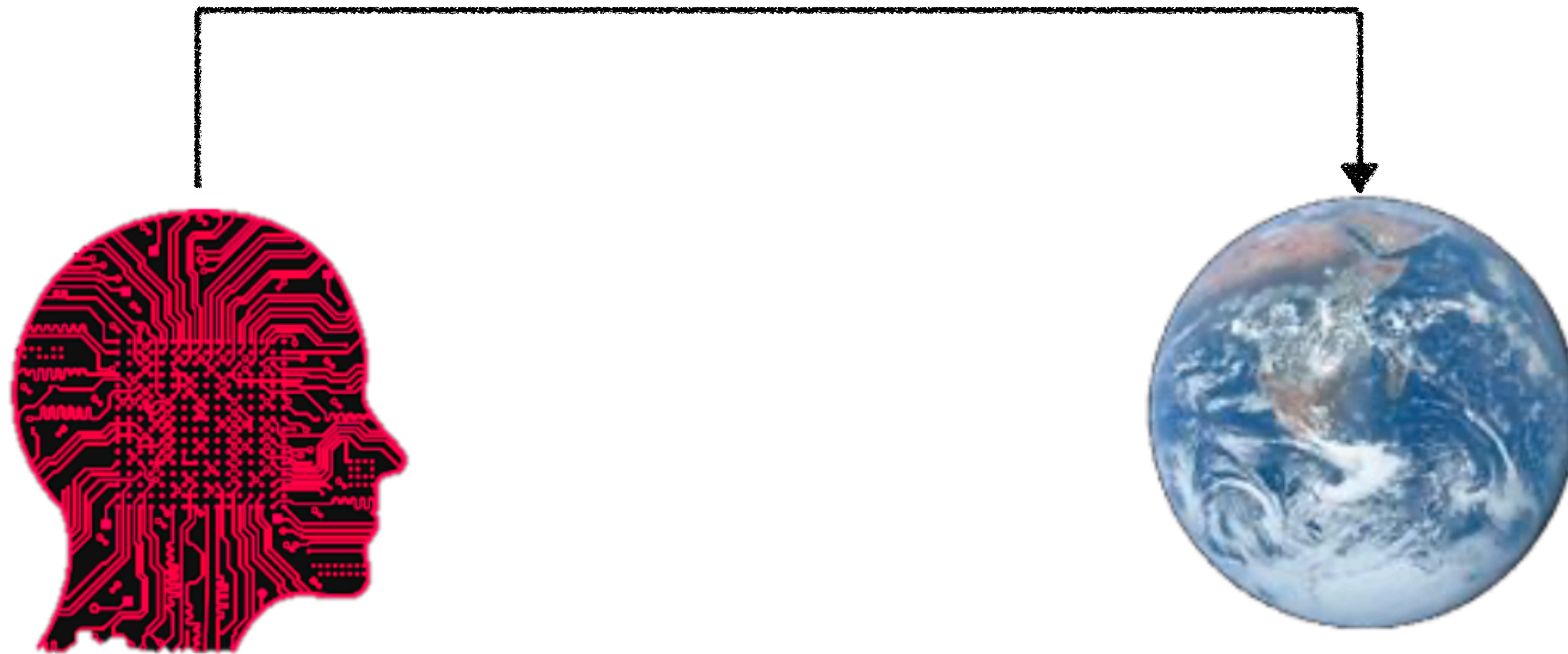
- A policy fully defines the behaviour of an agent
- It's a map from state to action
- Deterministic policy: $a = \pi(s)$
- Stochastic policy: $\pi(a | s) = P[A_t = a | S_t = s]$
- In general, one can view policy as some probability distribution, conditional on the state
 $\pi : \mathcal{S} \rightarrow \Delta(A)$



Sequential Decision Making

Learning to act by **sequentially** *interacting* with an **uncertain environment**

$$\text{Action: } a_t \sim \pi(\cdot, s_t)$$



Agent

Environment

Reward: $R_t = r(s_t, a_t)$

$$r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$$

State: $s_t \in \mathcal{S}$

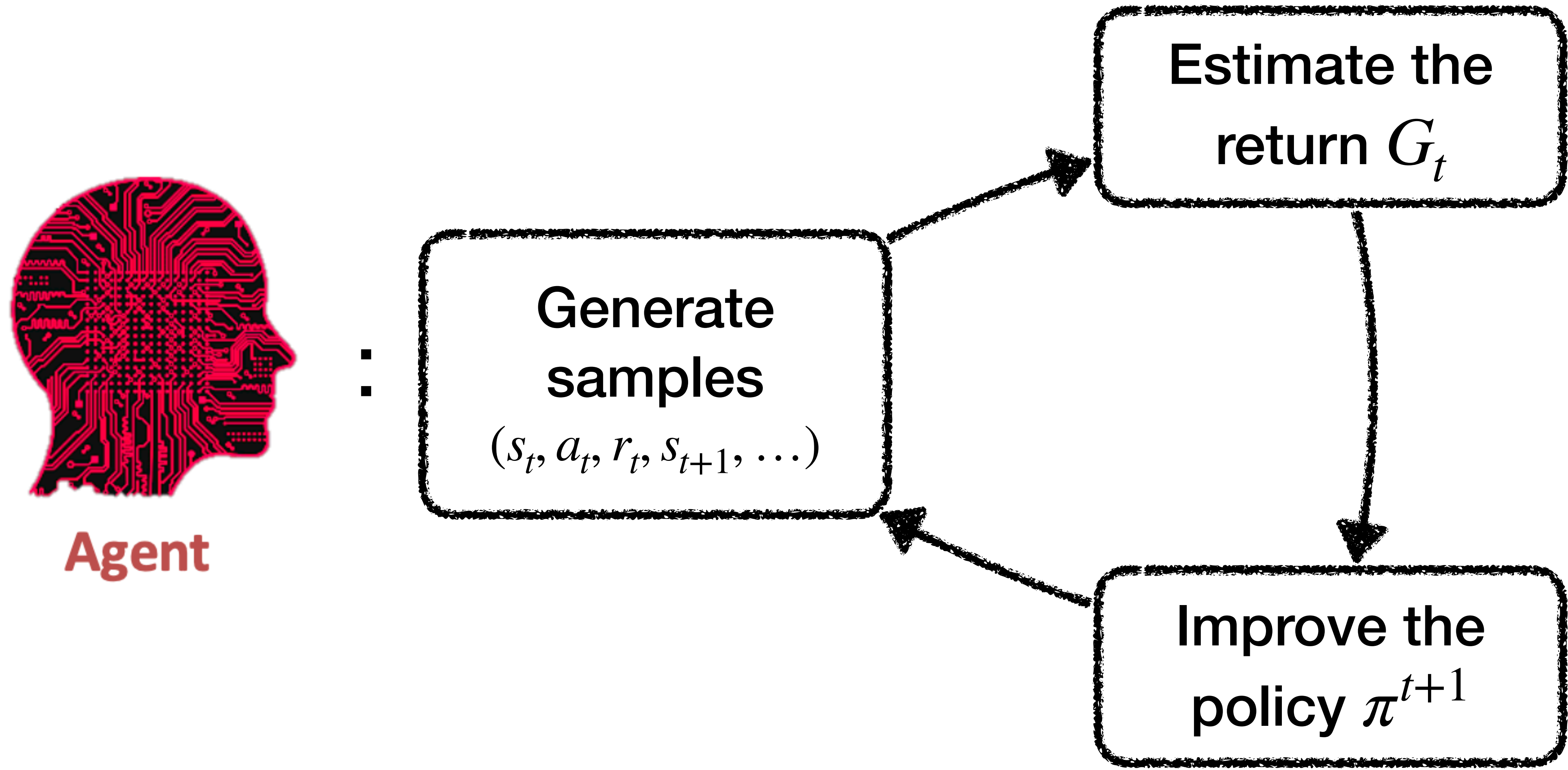
R_t

R_{t+1}

s_t

s_{t+1}

General Structure of an RL Agent Training

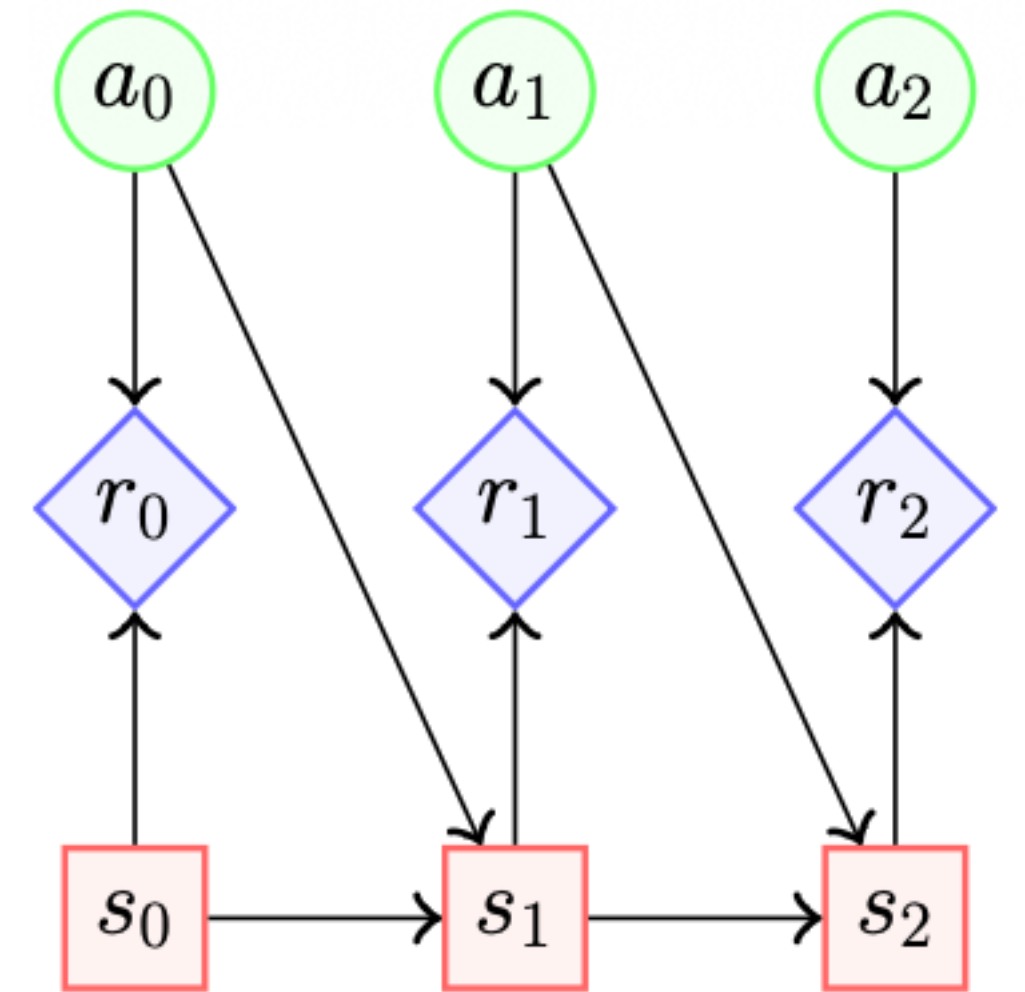


Markov Decision Process

Markov Decision Process

MDP:

1. $\mathcal{A}$ is an action space (discrete or continuous)
2. $\mathcal{S}$ is a state space (discrete or continuous)
3. Initial state distribution: $s_0 \sim p_0$
4. $p(r, s' | s, a) = \mathbb{P}(R_t = r, S_{t+1} = s' | S_t = s, A_t = a)$ is the joint probability of a reward r and next state s' , given a state s and action a
5. Discount factor $\gamma \in [0, 1]$



Markov property:

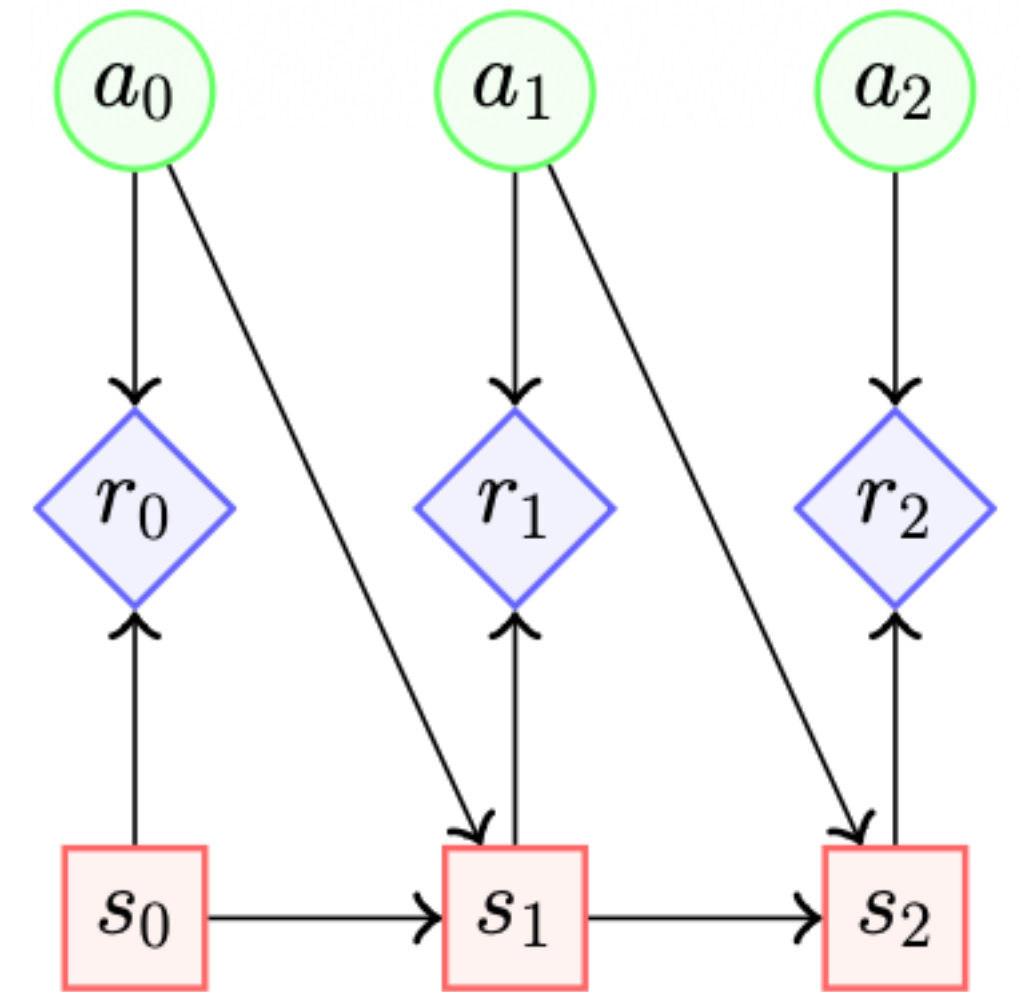
(the future is independent of the past given the present)

$$p(R_t, S_{t+1} | S_t, A_t, R_{t-1}, S_{t-1}, A_{t-1}, \dots) = p(R_t, S_{t+1} | S_t, A_t)$$

Markov Decision Process

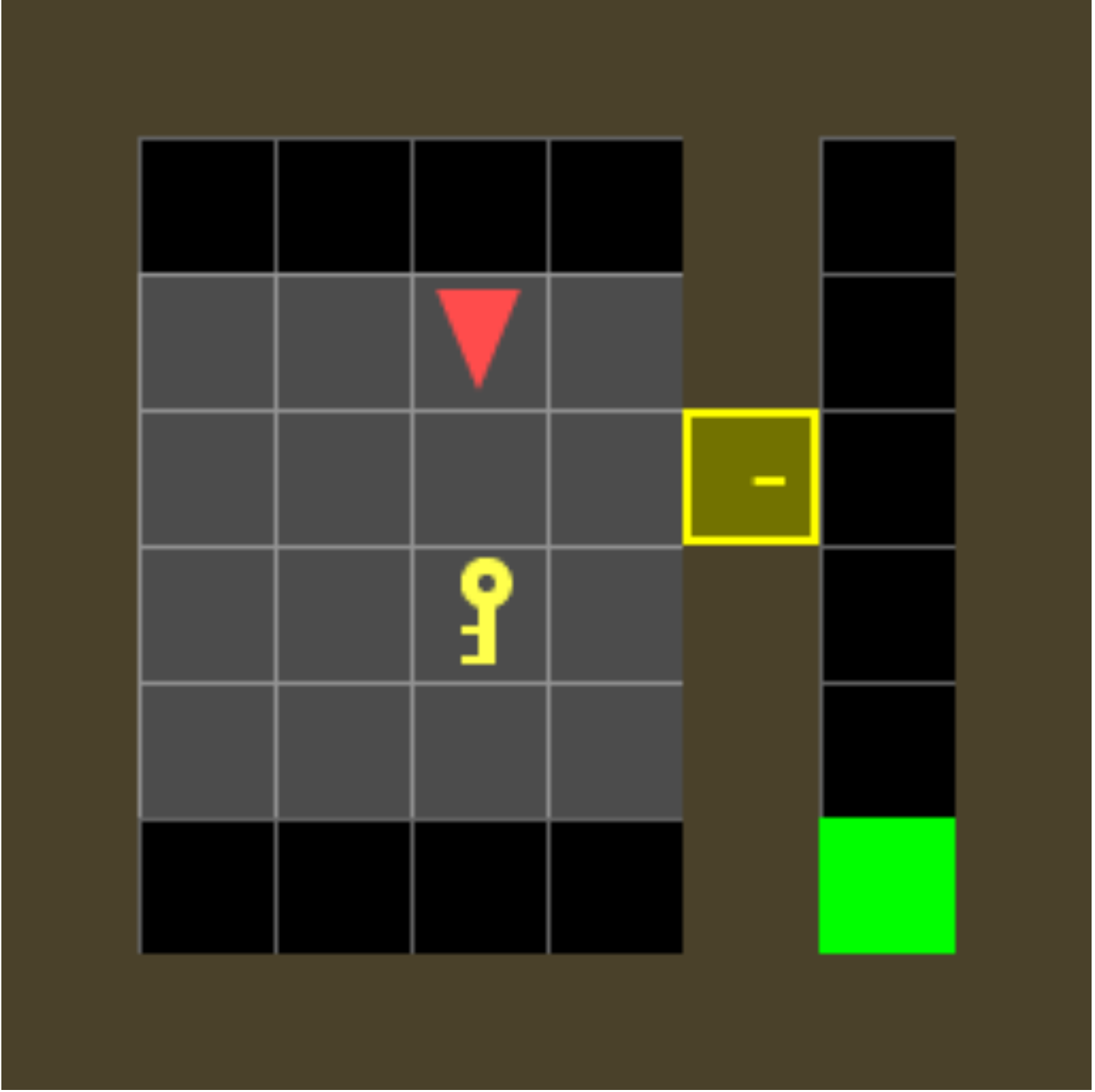
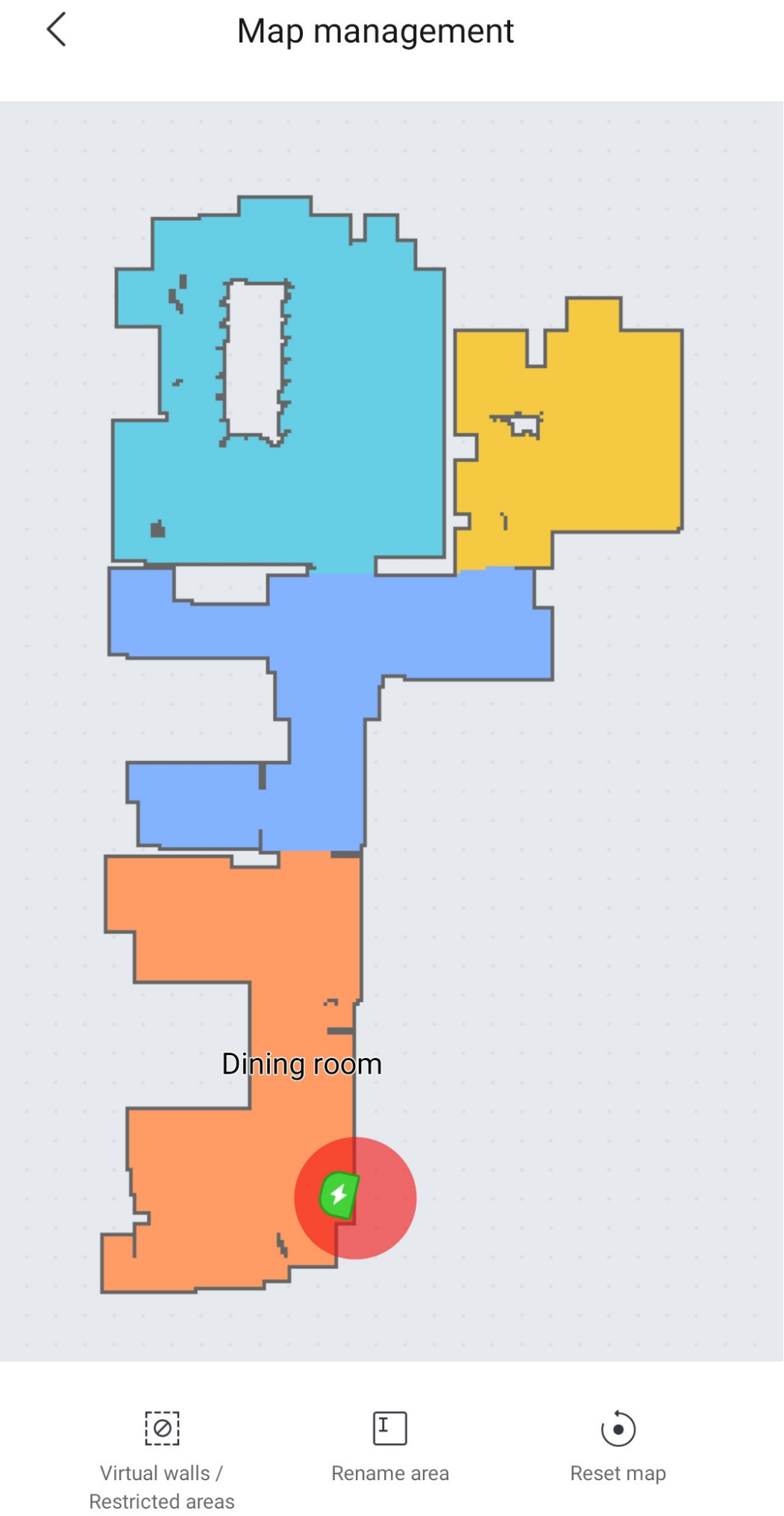
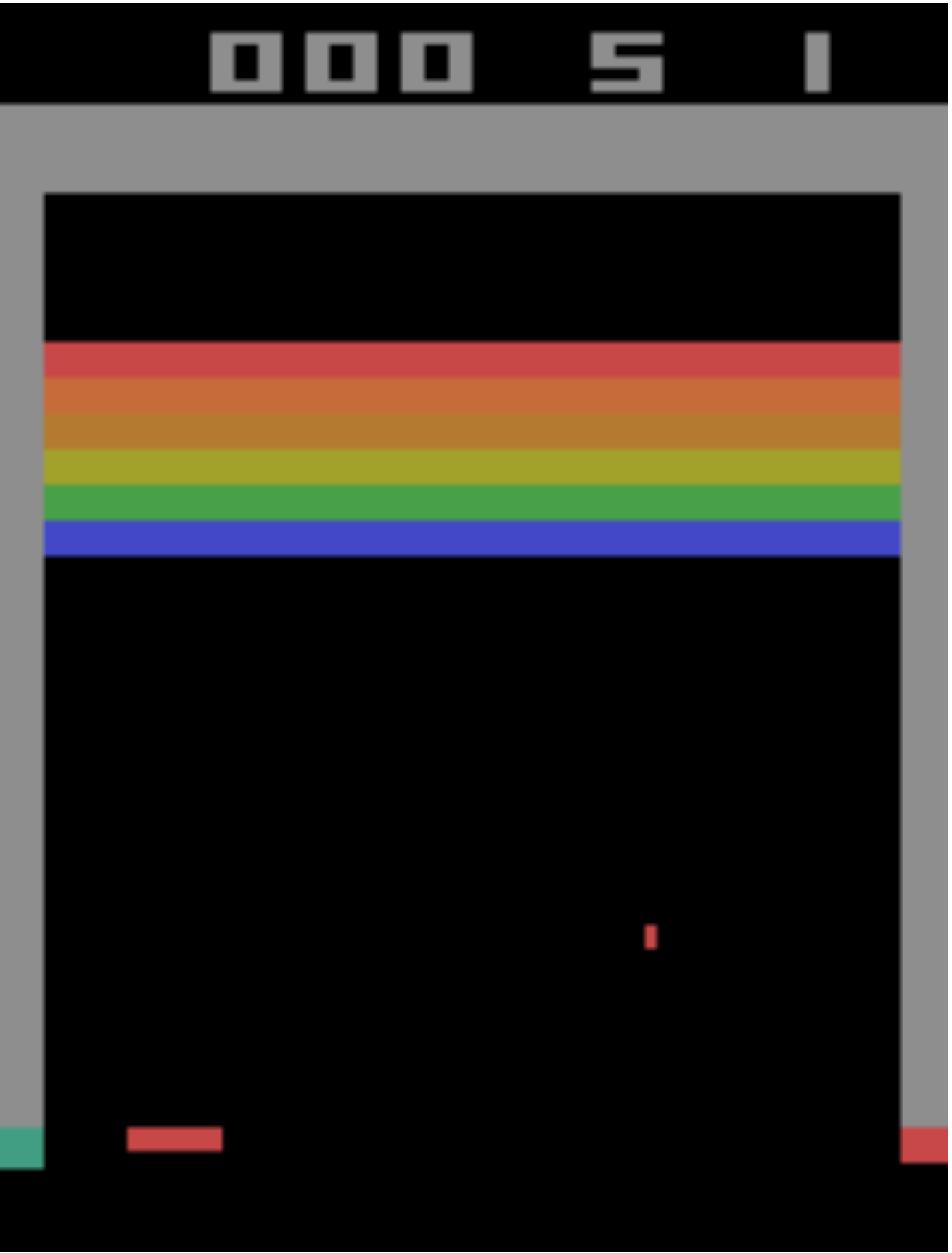
MDP:

1. $\mathcal{A}$ is an action space (discrete or continuous)
2. $\mathcal{S}$ is a state space (discrete or continuous)
3. Initial state distribution: $s_0 \sim p_0$
4. $p(r, s' | s, a) = \mathbb{P}(R_t = r, S_{t+1} = s' | S_t = s, A_t = a)$ is the joint probability of a reward r and next state s' , given a state s and action a
5. Discount factor $\gamma \in [0, 1]$

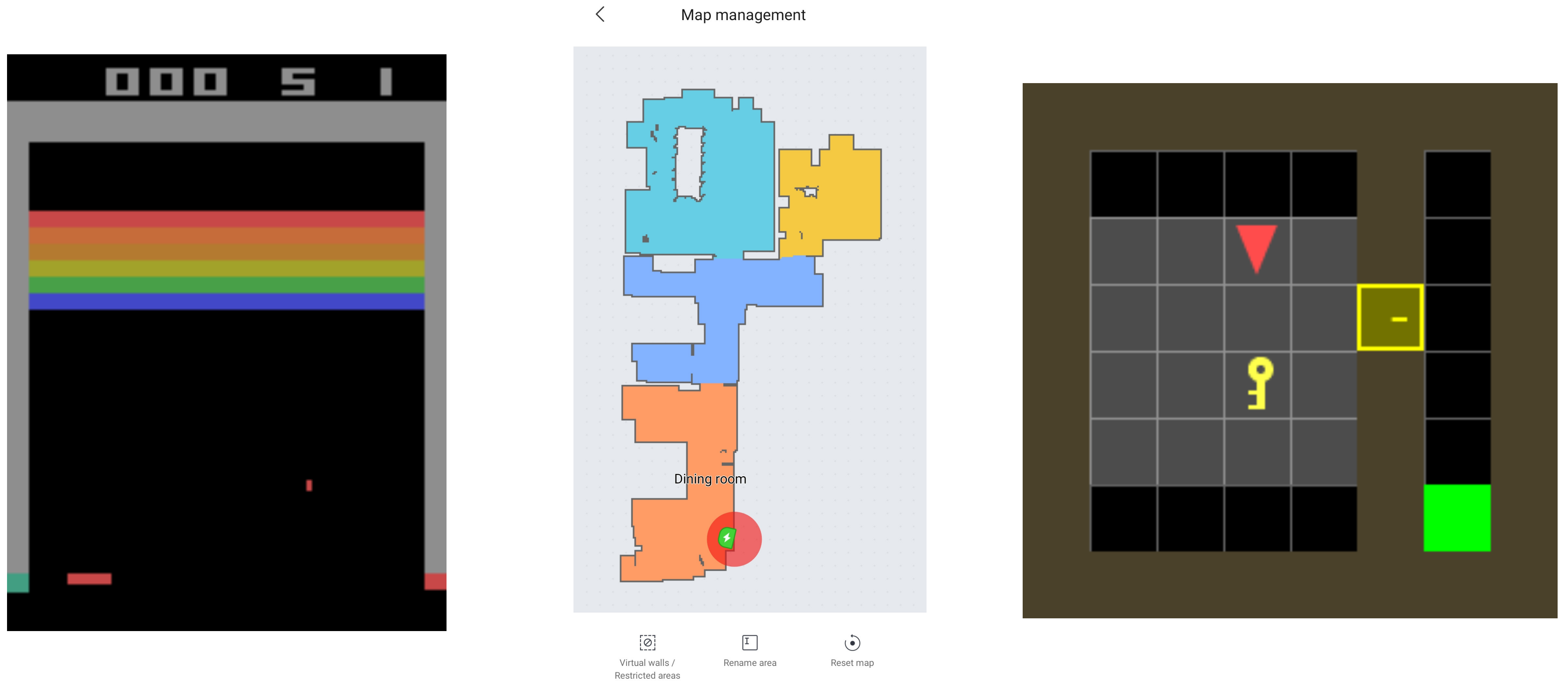


MDP $\approx$ controlled Markov Chain + Performance objective

Non-Markovian Environments



Non-Markovian Environments



Can be made MDP by proper state augmentation

Equivalent Definition

MDP is a 4-tuple $(\mathcal{S}, \mathcal{A}, p, r)$:

1. $\mathcal{A}$ is an action space

2. $\mathcal{S}$ is a state space

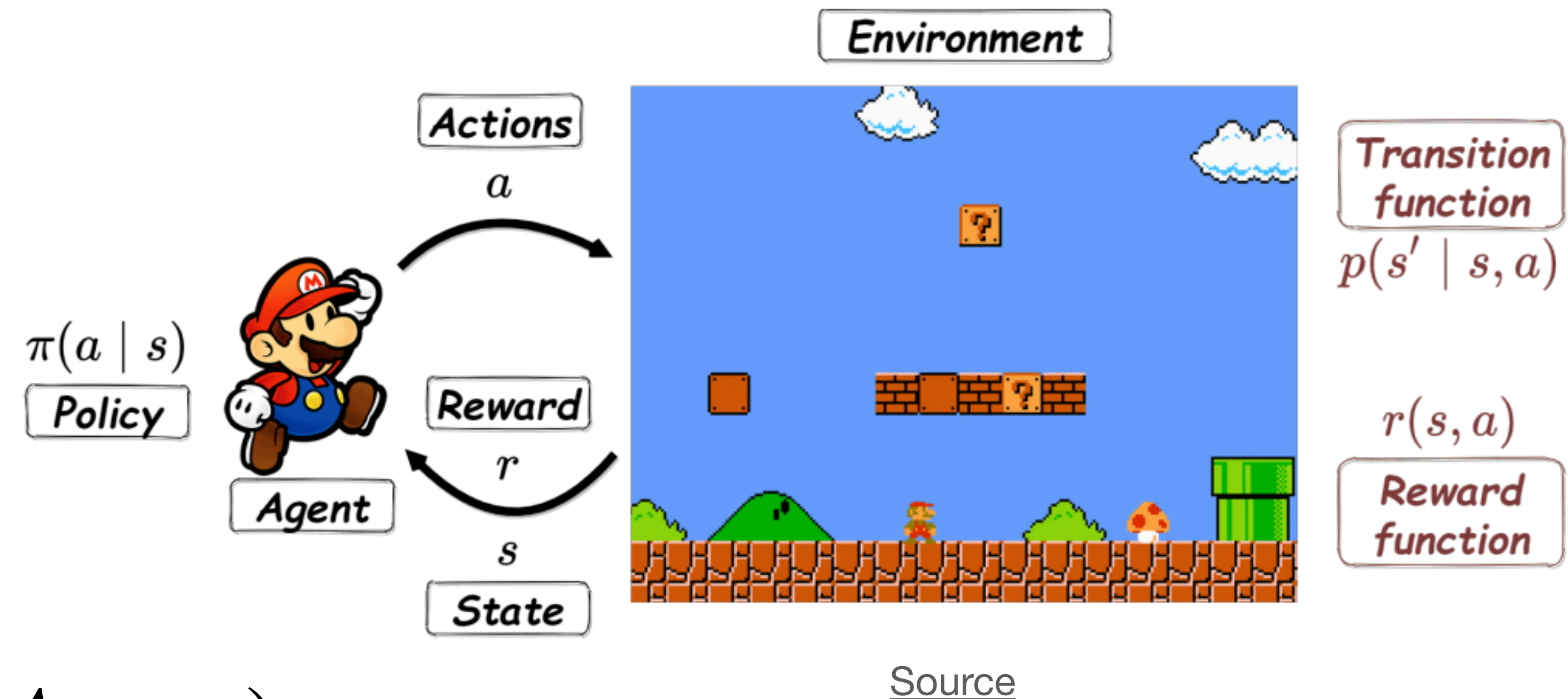
3. $P : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$

$p(s' | s, a) = \mathbb{P}(S_{t+1} = s' | S_t = s, A_t = a)$
is a state-transition function

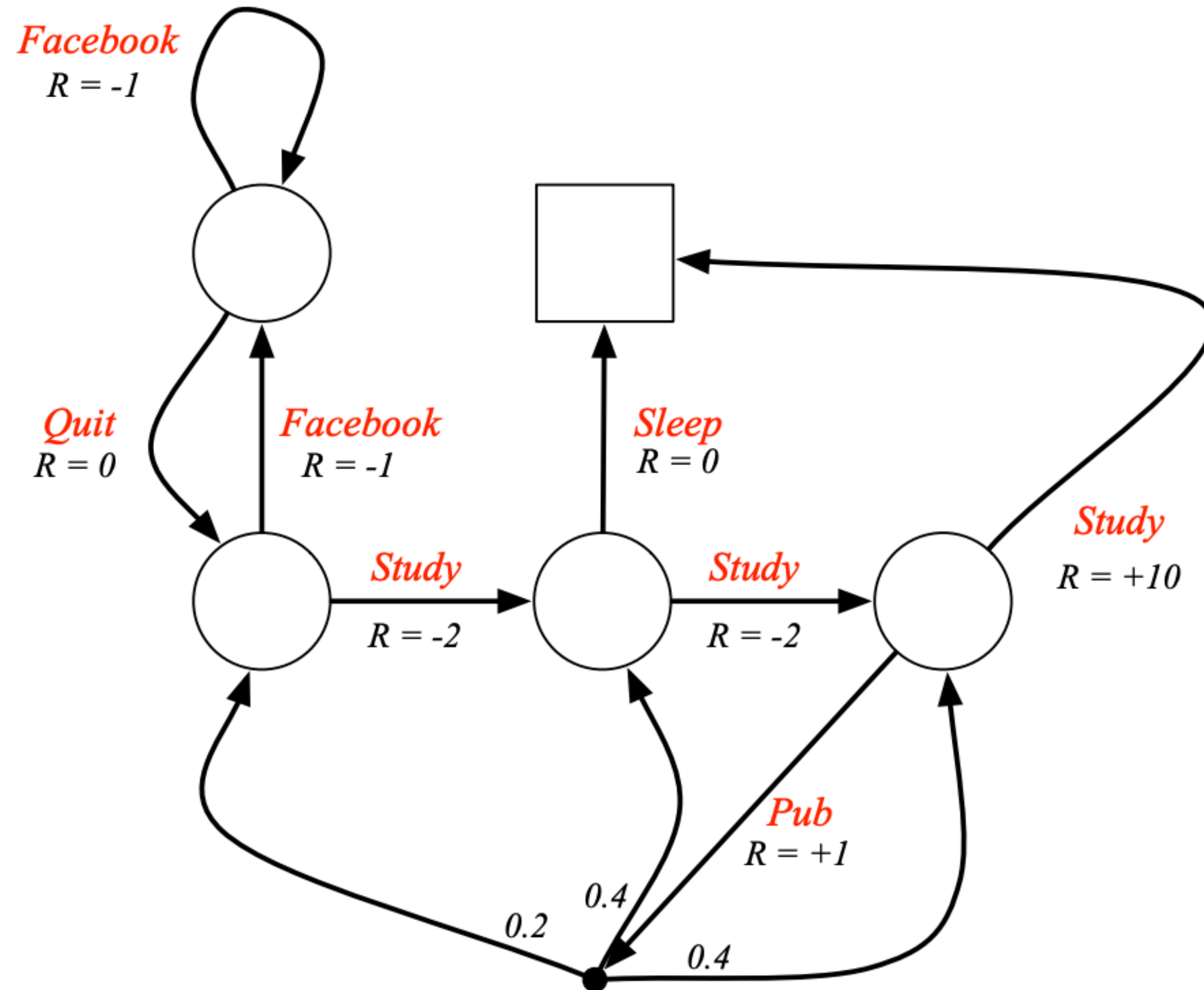
4. $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is a reward function

5. We aim to develop a policy

$\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$



Example: Student MDP

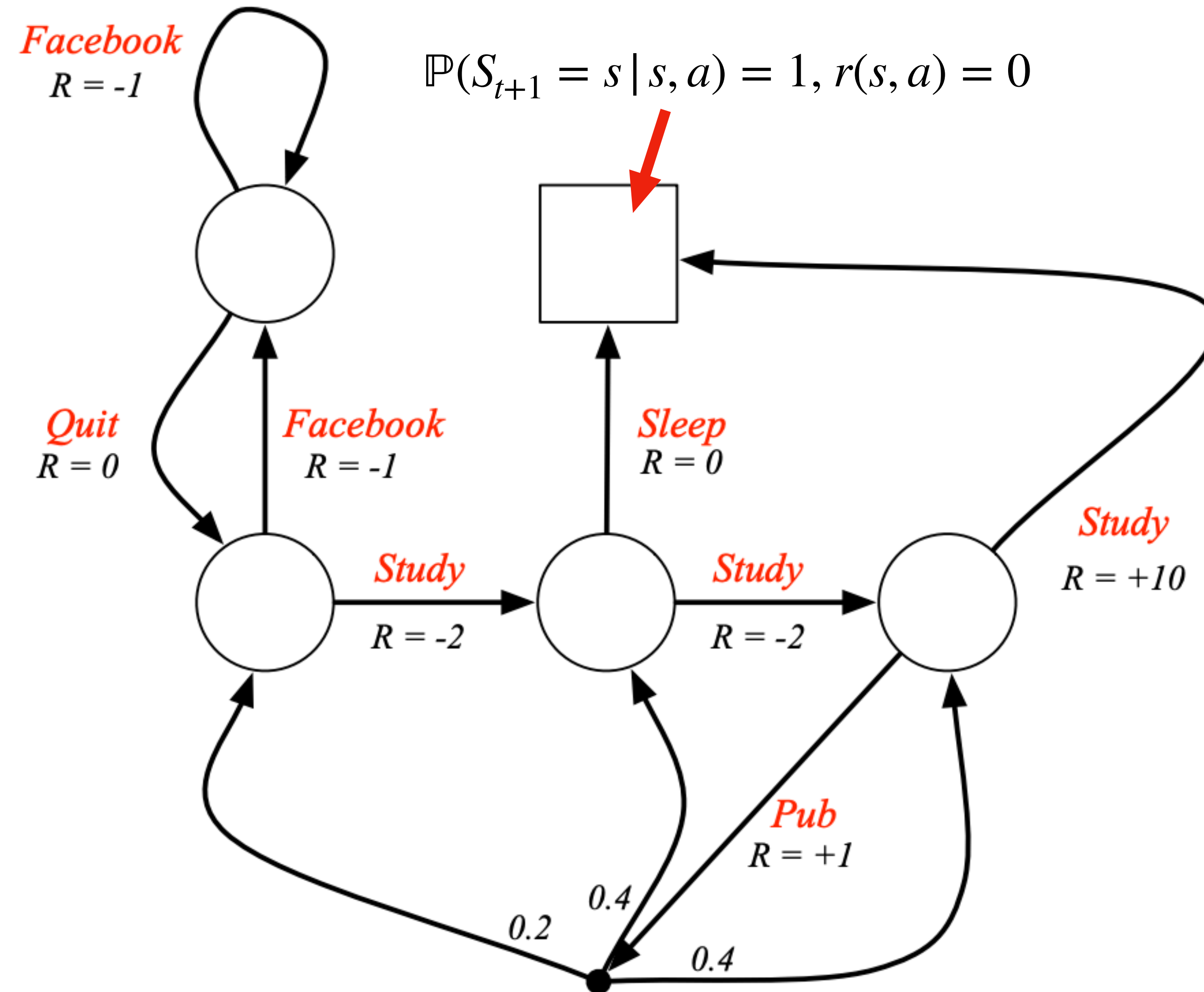


Example: Student MDP

Terminal state:

Tabular MDP:

$$|\mathcal{A}| < \infty, |\mathcal{S}| < \infty$$



Returns and Episodes

Let T is a final time step. If $T < \infty$ then environment is called *episodic*.

Cumulative reward is called a **return or reward-to-go**. Note that in general it is a random variable.

The diagram shows the equation for the cumulative reward G_t with several annotations:

- A blue arrow points from the text "Cumulative reward" to the term G_t , which is enclosed in a blue box.
- A green arrow points from the text "Immediate reward" to the term R_t , which is enclosed in a green box.
- An orange arrow points from the text "Discount factor" to the term γ , which is enclosed in an orange box.
- A red arrow points from the text "Episode length" to the term T in the summation limit, which is enclosed in a red box.

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots = \sum_{k=t}^T \gamma^{k-t} R_k$$

Either $T = \infty$ or $\gamma = 1$

Returns and Episodes

Let T is a final time step. If $T < \infty$ then environment is called *episodic*.

Cumulative reward is called a **return or reward-to-go**. Note that in general it is a random variable.

The diagram shows the equation for the cumulative reward G_t with several annotations:

- A blue arrow points from the text "Cumulative reward" to the G_t term, which is enclosed in a blue box.
- A green arrow points from the text "Immediate reward" to the R_t term, which is enclosed in a green box.
- An orange arrow points from the text "Discount factor" to the γ term in the second term, which is enclosed in an orange box.
- A red arrow points from the text "Episode length" to the T term in the summation limit, which is enclosed in a red box.

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots = \sum_{k=t}^T \gamma^{k-t} R_k$$

Important note: $G_t = R_t + \gamma(R_{t+1} + \gamma R_{t+2} + \dots) = R_t + \gamma G_{t+1}$

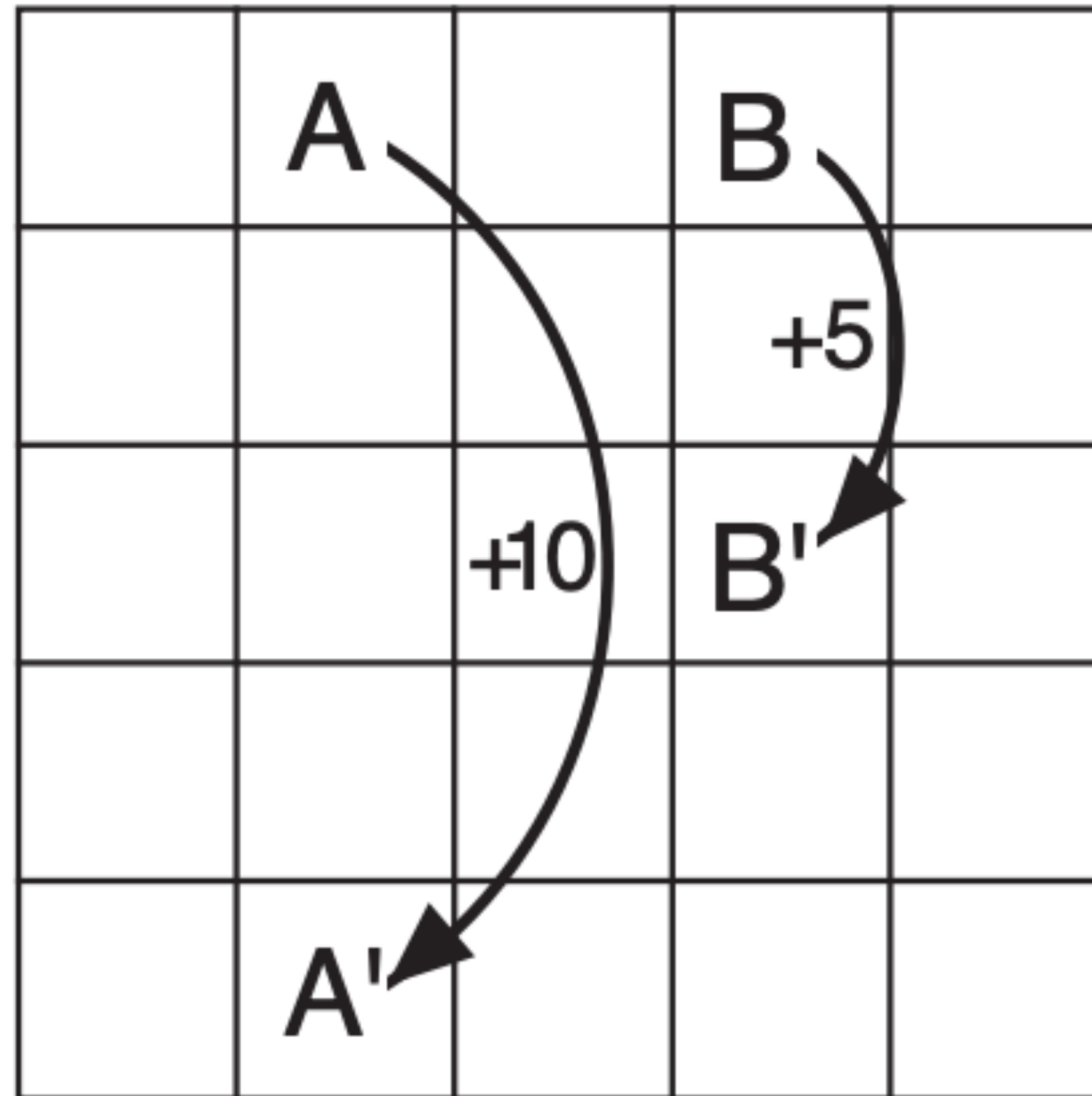
Objective Function

- Agent interacts with the environment following some policy $\pi(a \mid s)$.
- Policy stochasticity: $a \sim \pi(\cdot \mid s)$
- Environment stochasticity: $s' \sim p(\cdot \mid s, a)$
- A trajectory: $\tau = (s_0, a_0, s_1, \dots, s_{T-1}, a_{T-1}, s_T)$

$$\begin{aligned} J(\pi) &= \mathbb{E}_{\pi}[G_0] = \mathbb{E}_{p(\tau|\pi)}[G_0] = \\ &= \mathbb{E}_{s_0 \sim p_0}[\mathbb{E}_{a_0 \sim \pi(\cdot|s_0)}[\mathbb{E}_{s_1 \sim p(\cdot|s_0, a_0)}[r_0 + \gamma[\dots]]]] = \\ &= \mathbb{E}_{s_0}[\mathbb{E}_{a_0}[\mathbb{E}_{s_1}[r_0 + \gamma[\dots]]]] \rightarrow \max_{\pi} \end{aligned}$$

Environment Example

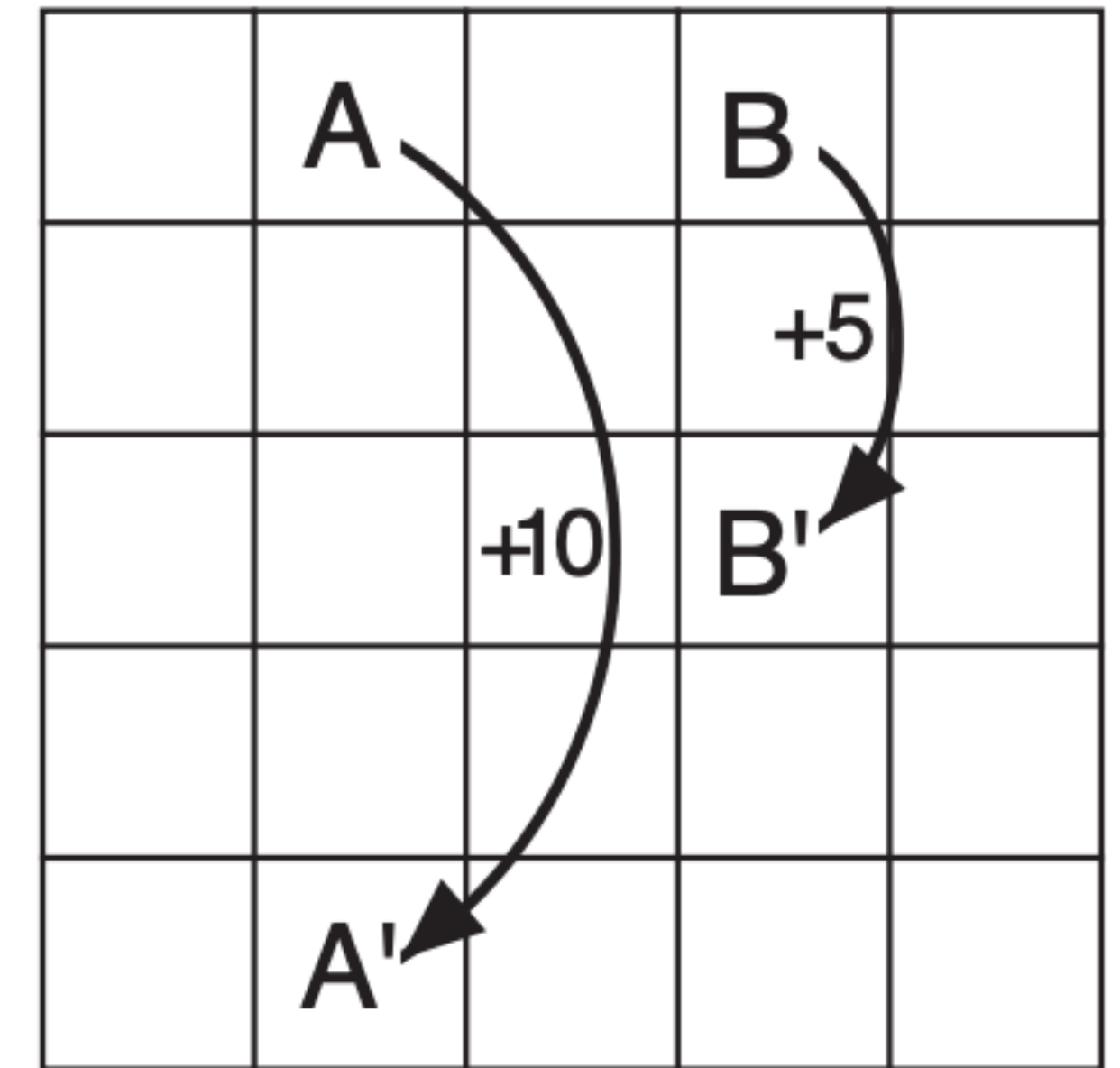
Grid World Example (Sutton & Barto):



For derivation check Sutton & Barto.
Page 72, Example 3.8: Gridworld

Simple Idea

- For every policy π compute $J(\pi)$
- Choose $\pi^* = \operatorname{argmax}_{\pi} J(\pi)$



Simple Idea

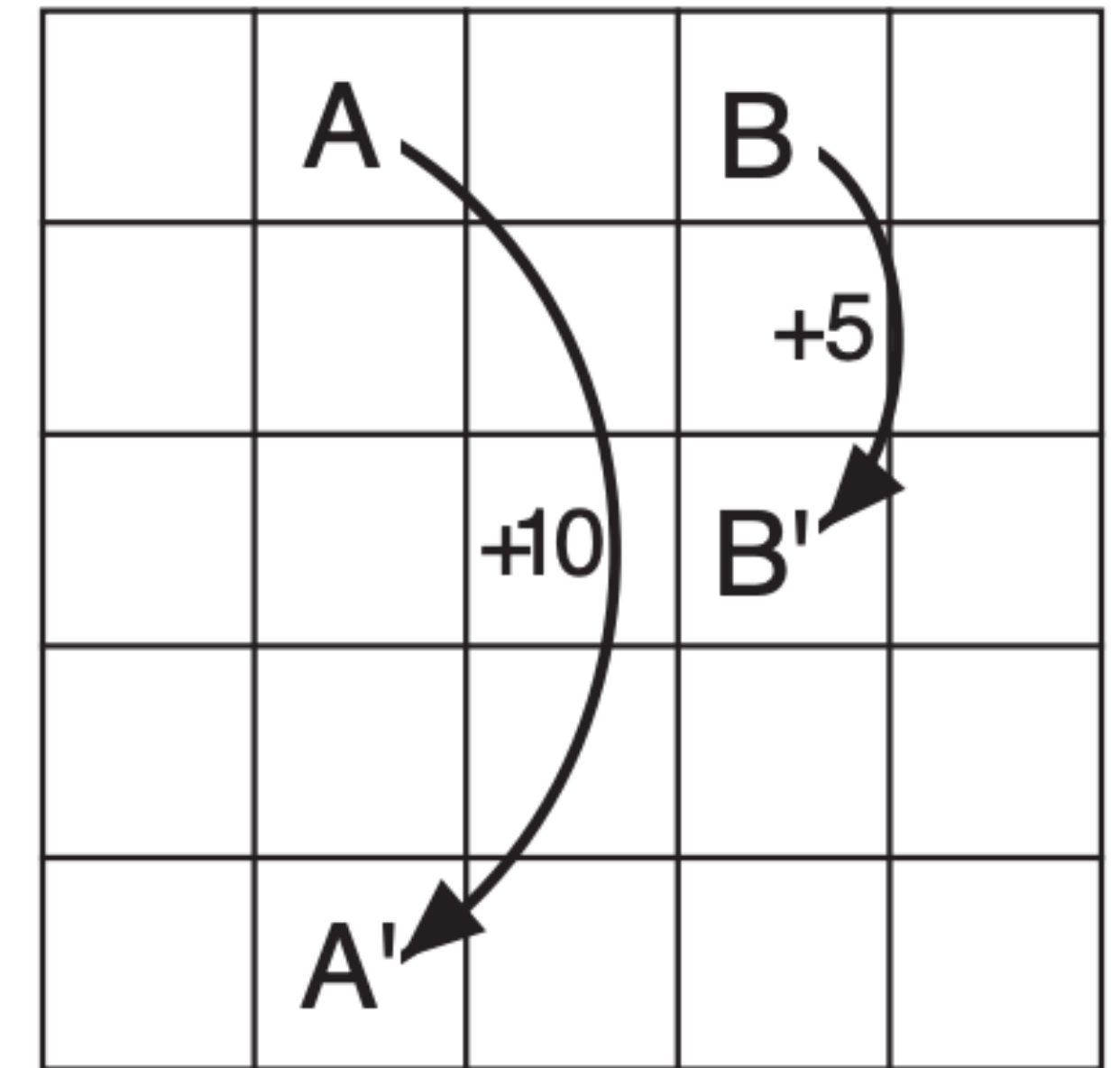
- For every policy π compute $J(\pi)$
- Choose $\pi^* = \operatorname{argmax}_{\pi} J(\pi)$

Exponentially many candidate policies:

$$|\mathcal{A}| \times |\mathcal{A}| \times \dots \times |\mathcal{A}| = |\mathcal{A}|^{|\mathcal{S}|}$$

Ways to
act in s_1

Ways to
act in s_2



State-value Function

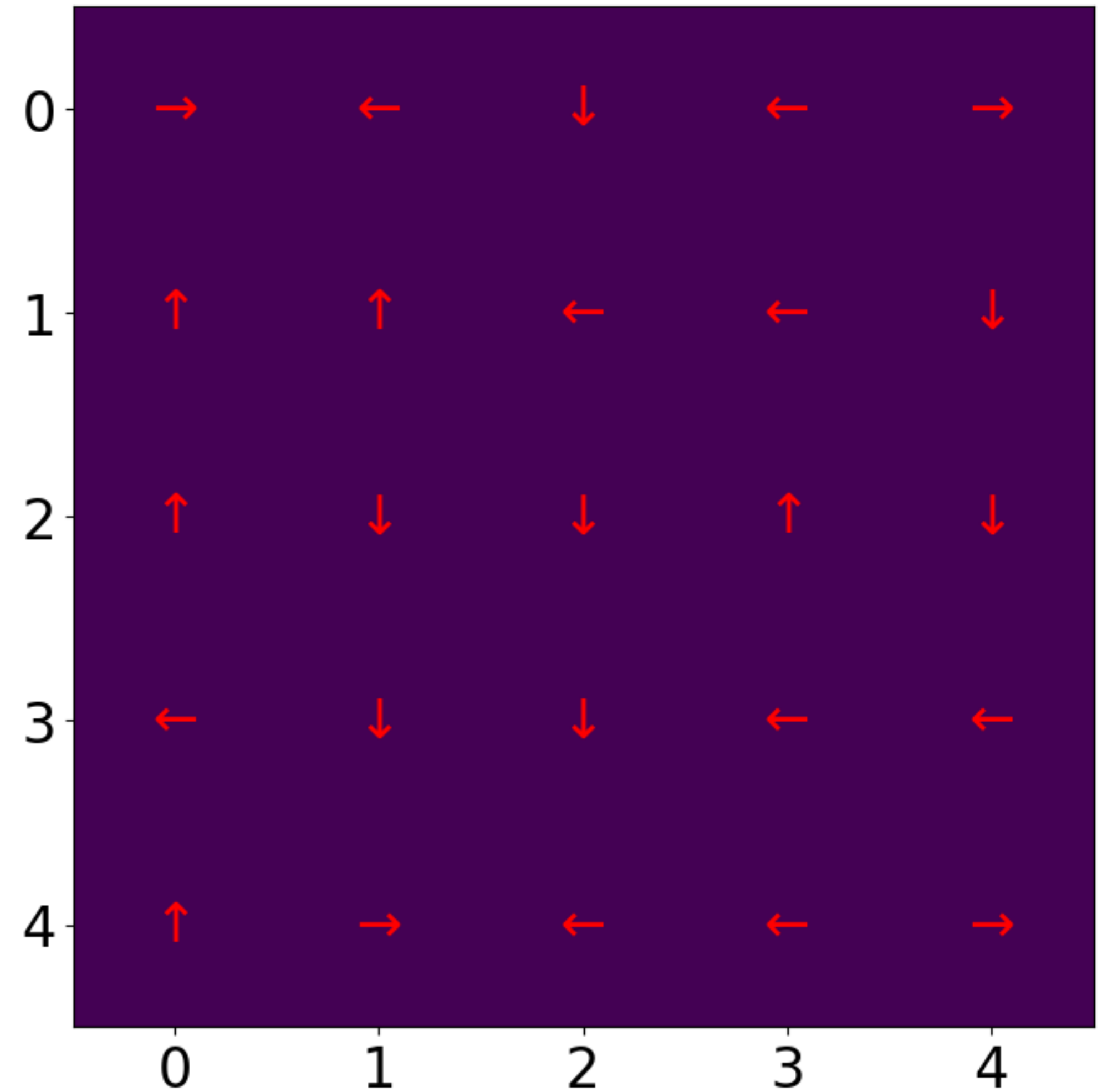
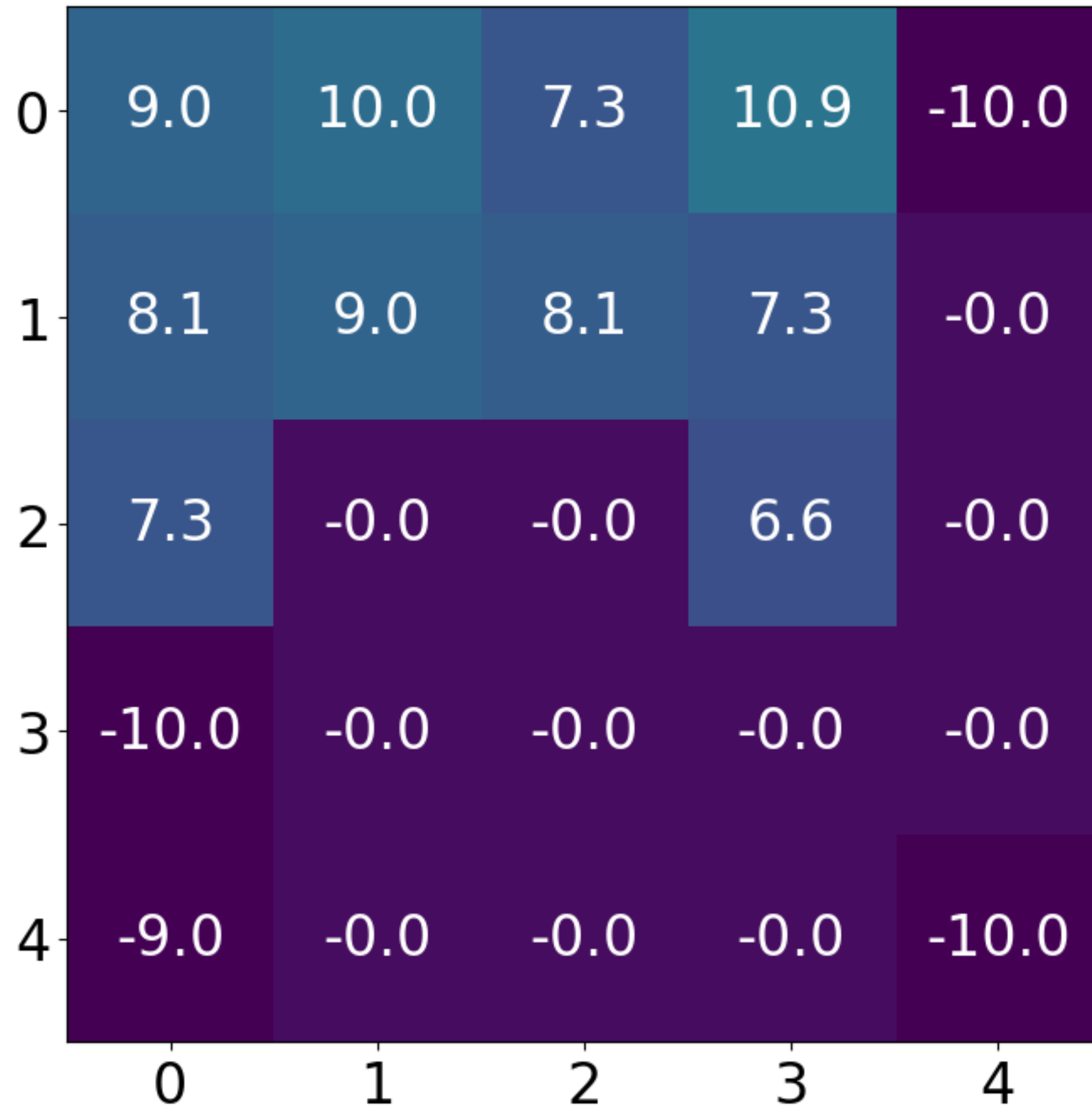
Estimate how good it is for an agent to be in a given state s and follow a policy π :

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi\left[\sum_{k=0}^T \gamma^k R_{t+k} | S_t = s\right]$$

If s is a terminal state then $V^\pi(s) = 0$.

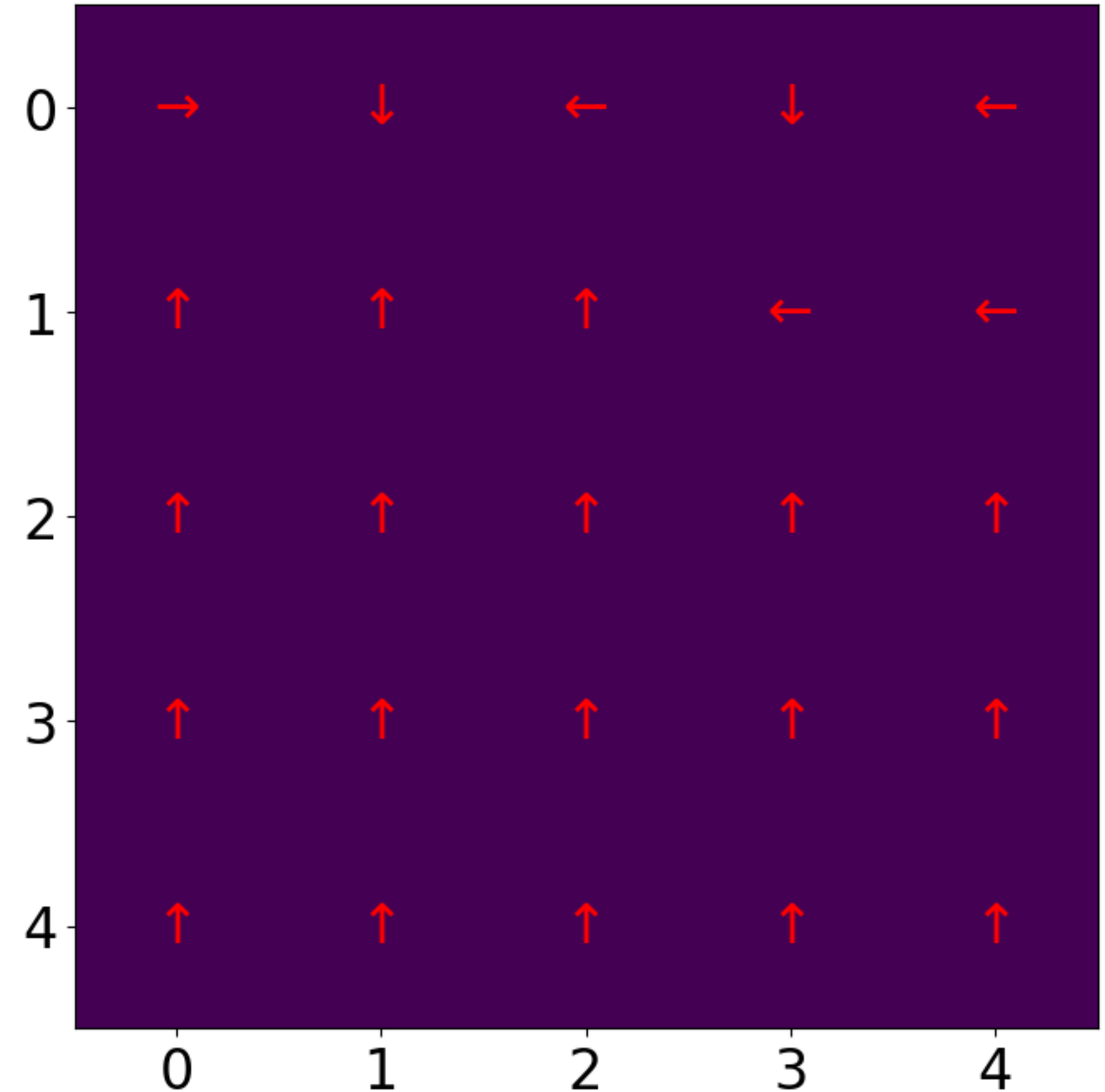
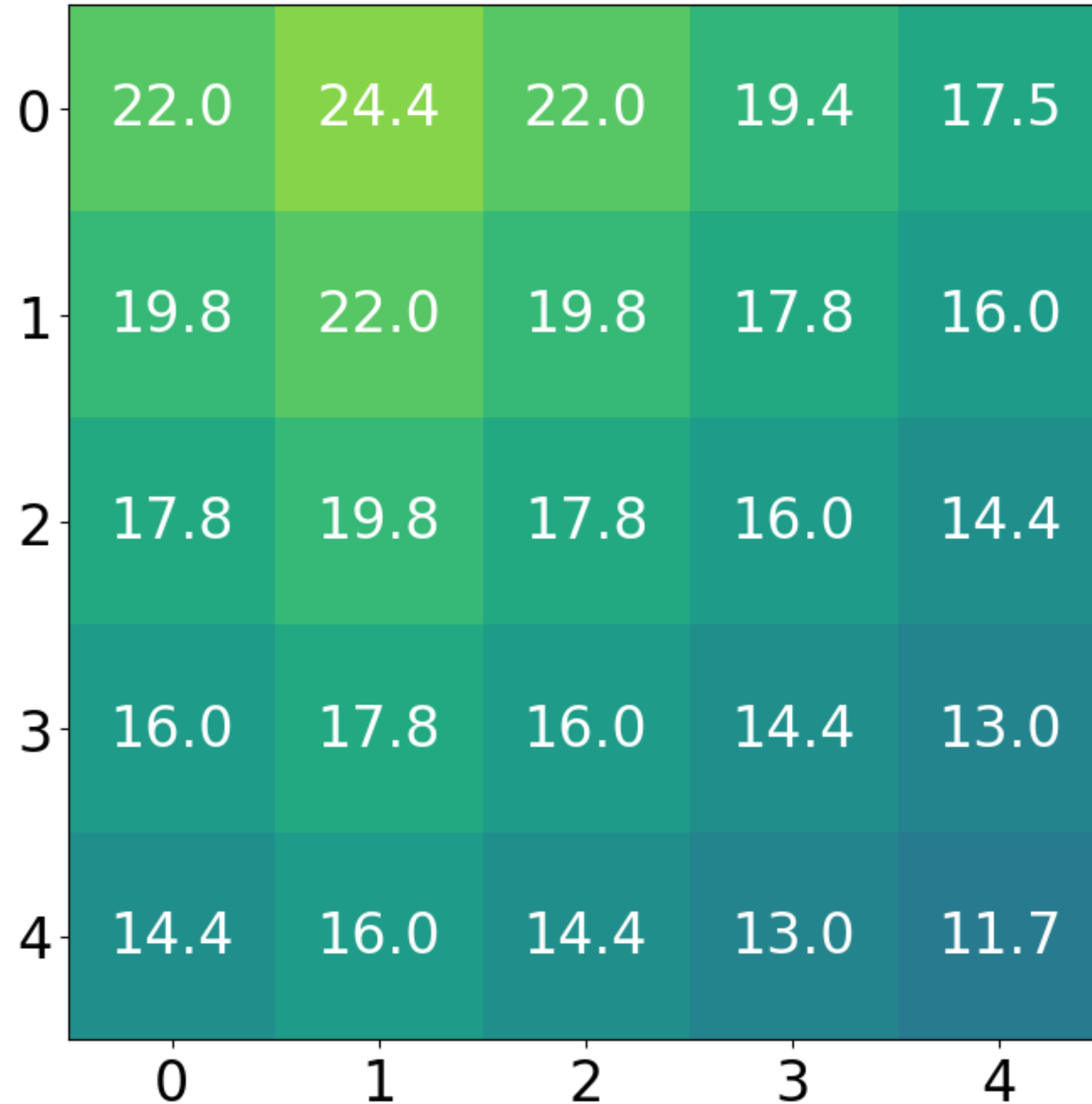
State-value Function

Grid World Example (Sutton & Barto) — Random Policy:



State-value Function

Grid World Example (Sutton & Barto) — Optimal Policy:



Action-value Function

Estimate how good it is for the agent to take an action a being in a given state s and follow a policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] = \mathbb{E}_\pi\left[\sum_{k=0}^T \gamma^k R_{t+k} | S_t = s, A_t = a\right]$$

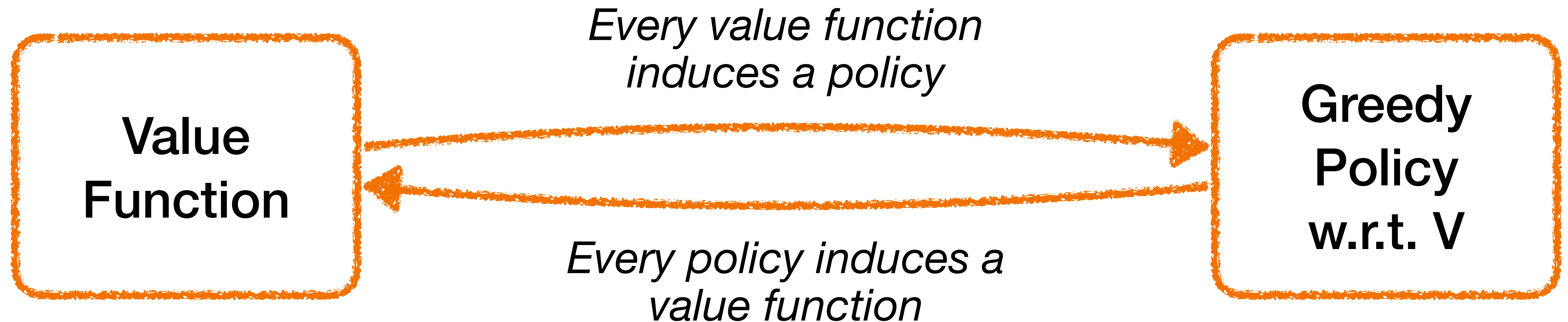
If s is a terminal state then $Q^\pi(s, a) = 0 \forall a \in \mathcal{A}$.

Intuition for $Q^\pi(s, a)$:

what if we choose now action a and act according to the policy π thereafter

Bellman Equations

Value Function and Policy

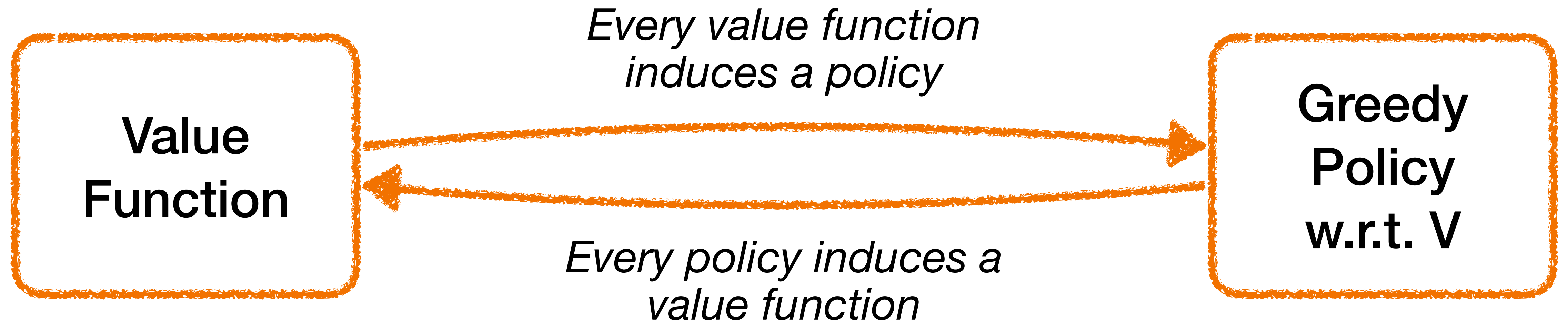


Theorem (Bellman):

Policy is optimal $\iff$ it is greedy w.r.t. its induced Value function

$$V^*(s) = \max_a [r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^*(s')]$$

Value Function and Policy

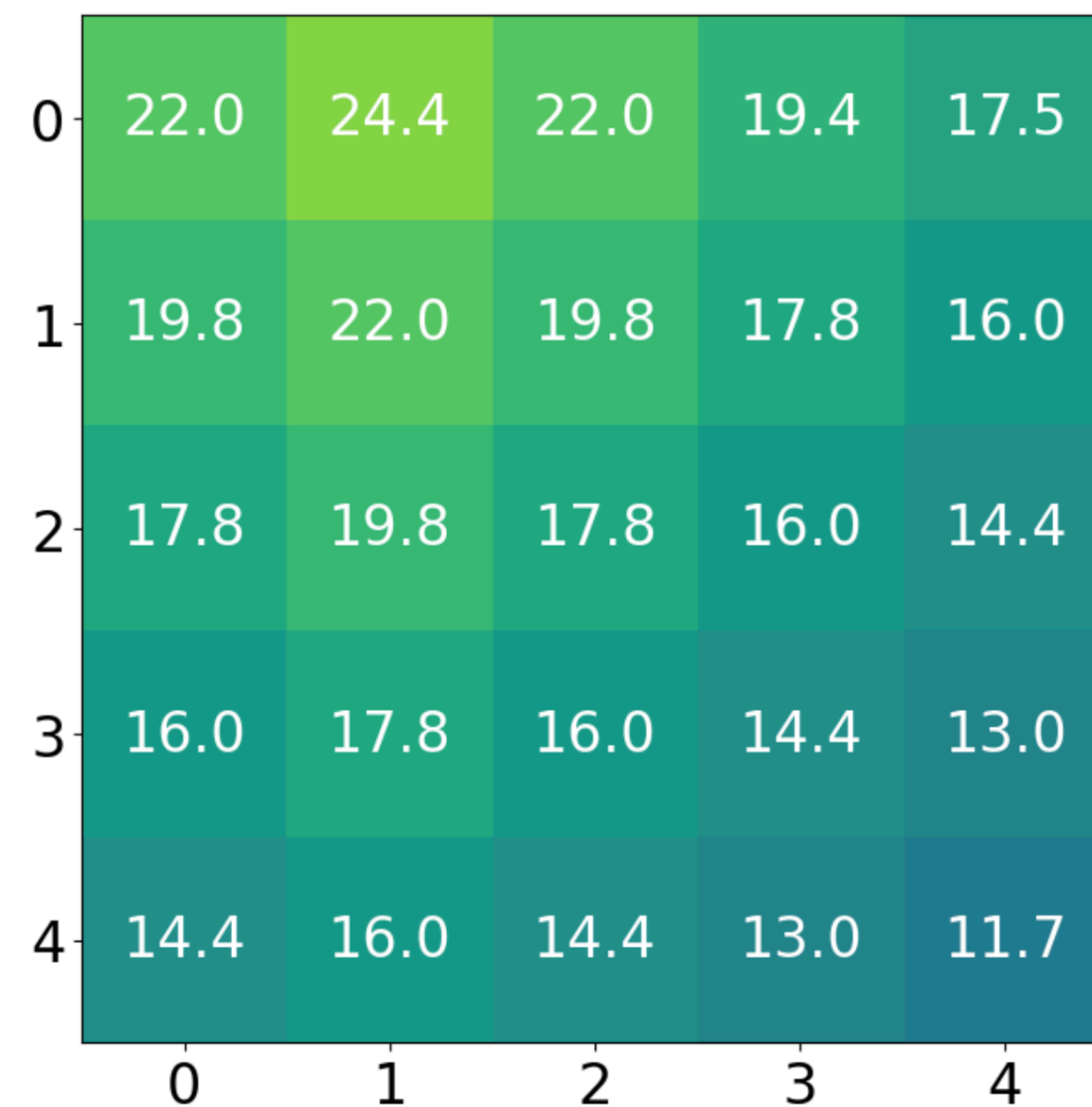


Theorem (Bellman):

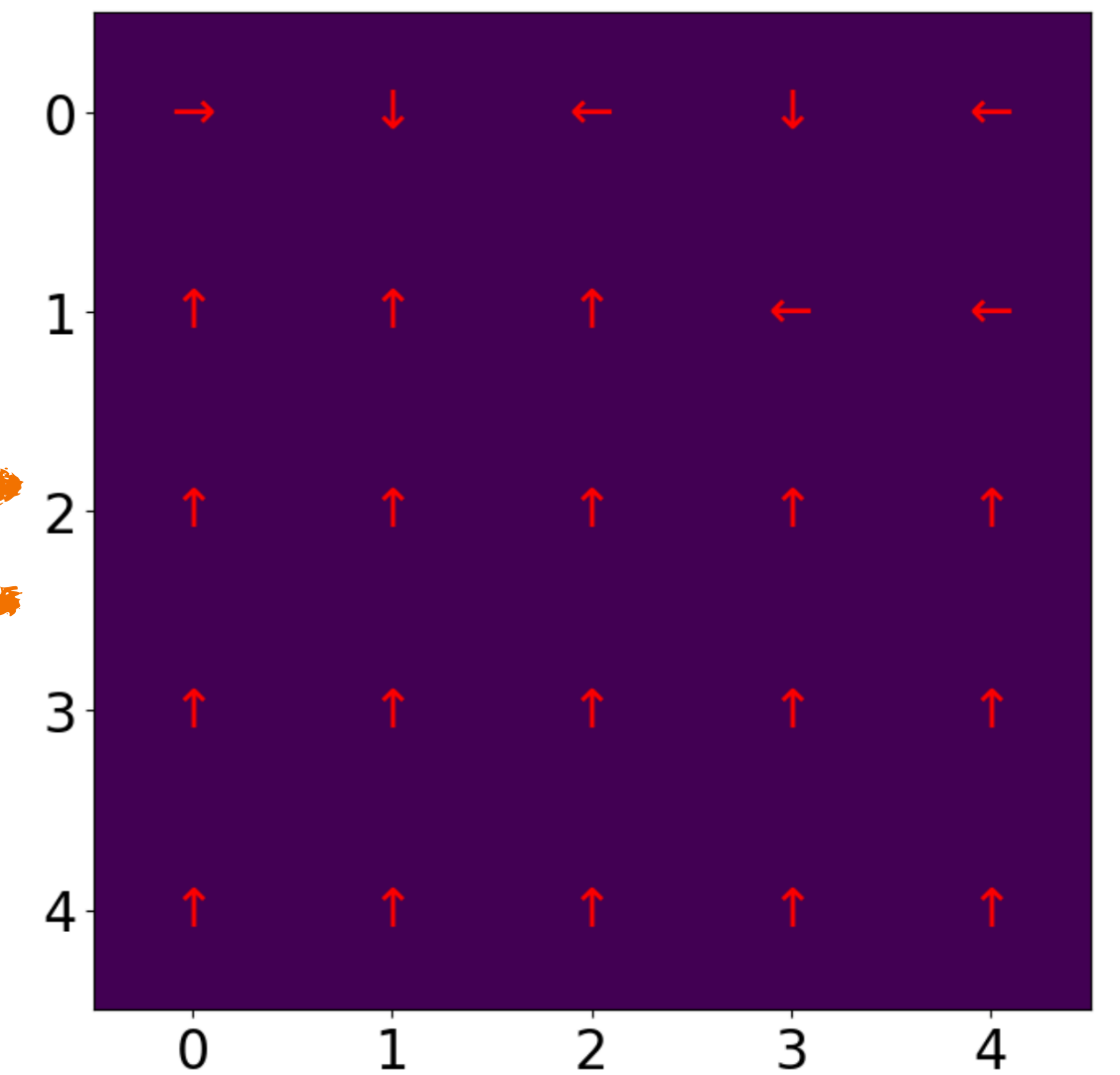
Policy is optimal $\iff$ it is greedy w.r.t. its induced Value function

$$\pi^*(s) \in \operatorname{argmax}_a [r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^*(s')] = \operatorname{argmax}_a Q^*(s, a)$$

Value Function and Policy



*Every value function
induces a policy*



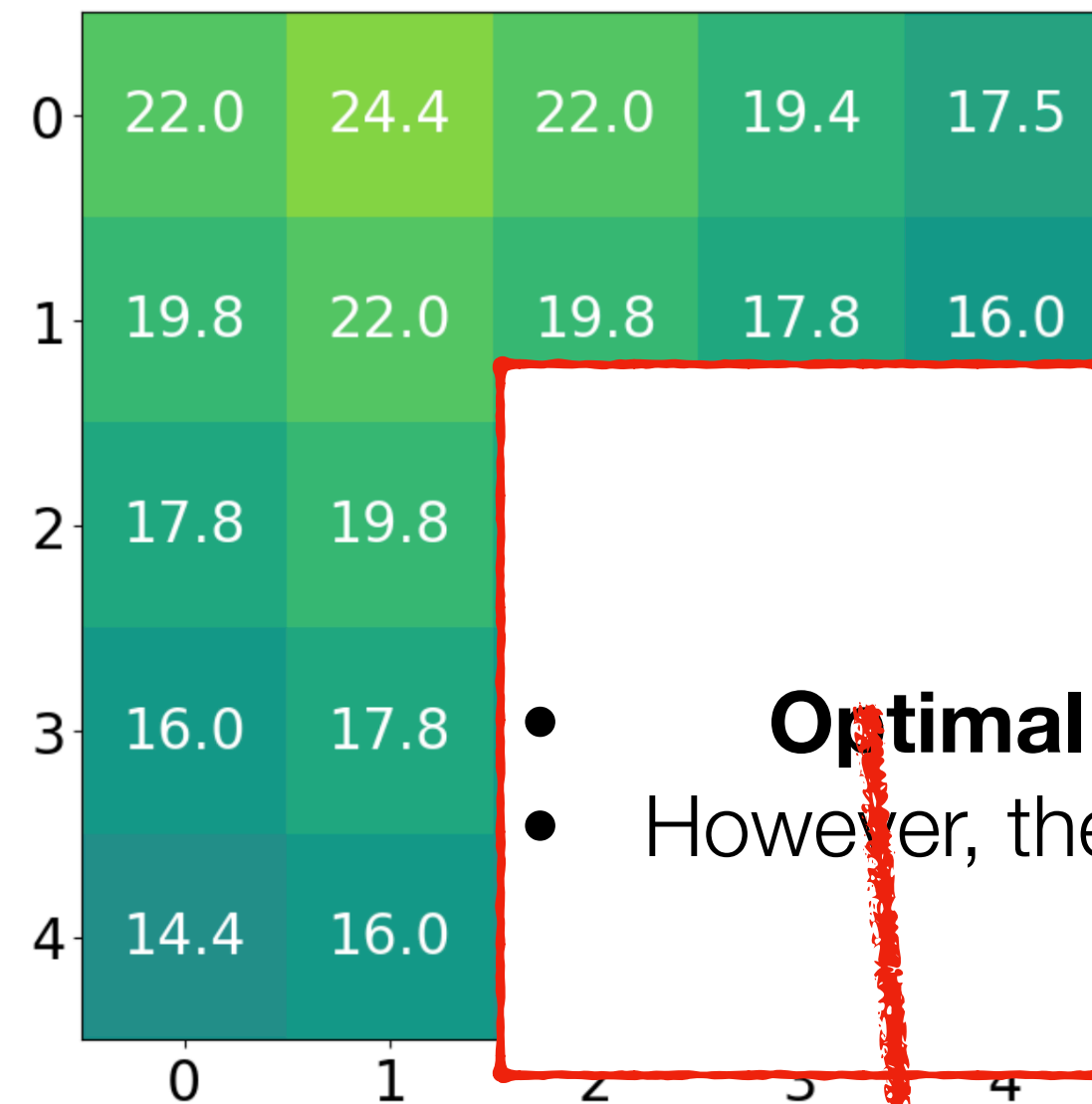
*Every policy induces a
value function*

Theorem (Bellman):

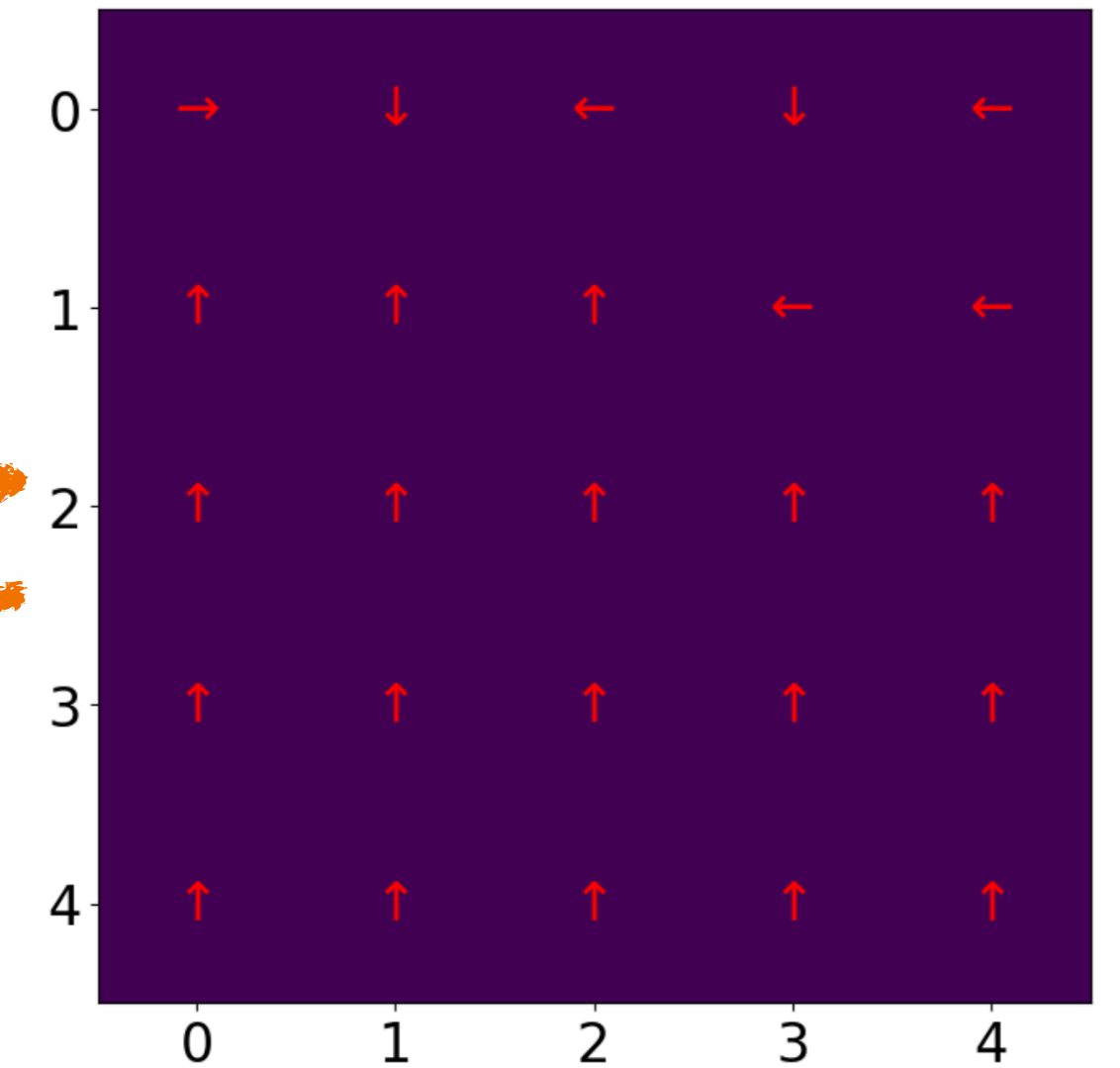
Policy is optimal $\iff$ it is greedy w.r.t. its induced Value function

$$V^*(s) = \max_a [r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^*(s')]$$

Value Function and Policy



Every value function induces a policy



Important Note:

- **Optimal Value** function **is unique** for the MDP
- However, the **optimal policy** need not to be unique

Theorem (Bellman):

Policy is optimal $\iff$ it is greedy w.r.t. its induced Value function

$$V^*(s) = \max_a [r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^*(s')]$$

Assumptions

MDP: we considered here the setting, where we have **uncertainty in how to act**, but **fully observable state**

POMDP: the assumption can be relaxed by considering O_t as a noisy version of the true state S_t^e

Assumptions

MDP: we considered here the setting, where we have **uncertainty in how to act**, but **fully observable state**

POMDP: the assumption can be relaxed by considering O_t as a noisy version of the true state S_t^e

Assumptions in the example: we have access to the transition dynamics, reward and full state observation

Practical RL: how to act, if we do not have access to the model of the environment

(Contextual) Bandits

Let's Take A Step Back...

Initially, we treated RL problem differently because of three main pillars:

- Reward Function (Feedback) instead of directly provided correct labels
- Exploration Vs Exploitation
- Our actions affect the data we collect

Theorem (Bellman):

Policy is optimal $\iff$ it is greedy w.r.t. its induced Value function

$$\pi^*(s) \in \operatorname{argmax}_a [r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^*(s')] = \operatorname{argmax}_a Q^*(s, a)$$

Let's Take A Step Back...

Initially, we treated RL problem differently because of three main pillars:

- Reward Function (Feedback) instead of directly provided correct labels
- Exploration Vs Exploitation
- ~~Our actions affect the data we collect~~ $\Rightarrow$

Bandits

Theorem (Bellman):

Policy is optimal $\iff$ it is greedy w.r.t. its induced Value function

$$\pi^*(s) \in \operatorname{argmax}_a [r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^*(s')] = \operatorname{argmax}_a r(s, a)$$

Bandits: Problem Formulation

- We start from some initial state $s_t \sim \rho(\cdot | X)$ (can be fixed $s \sim \delta_{s_0}$)
- We get some observation $x_t \sim \mu(\cdot | s_t)$
- We do not know the true d.g.p. behind the reward function $r(s_t, a_t)$
- Our task is to choose an action to maximize G_0 from **repeated experiments**

$$\pi^*(s) \in \operatorname{argmax}_a [r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^*(s')] = \operatorname{argmax}_a r(s, a)$$
$$\hat{\pi}(s) \in \hat{r}(s, a)$$

Bandits: Problem Formulation

Q: So, is it just a classical supervised learning?

Bandits: Problem Formulation

Q: So, is it just a classical supervised learning?

Not quite:

- We still have **reward function**

Bandits: Problem Formulation

Q: So, is it just a classical supervised learning?

Not quite:

- We still have **reward function**
- Still need to collect the samples ourselves => **exploration-exploitation trade-off**

Two Ways to View Bandits

1. Repeated One-Stage RL:

- We have interactions with the environment that end after first round (all actions transition us into terminal state directly);
- Our goal is long-term average reward from these repeated experiments.

2. Action-Independent State Transitions:

- We have a classical RL setting, but our actions do not affect state transitions;
- One can view this as state transitions are generated by the environment only, while our actions affect only the immediate reward.

Idea: Mathematical treatment of both approaches is identical, but one can recognize Bandit tasks in a lot of different applications

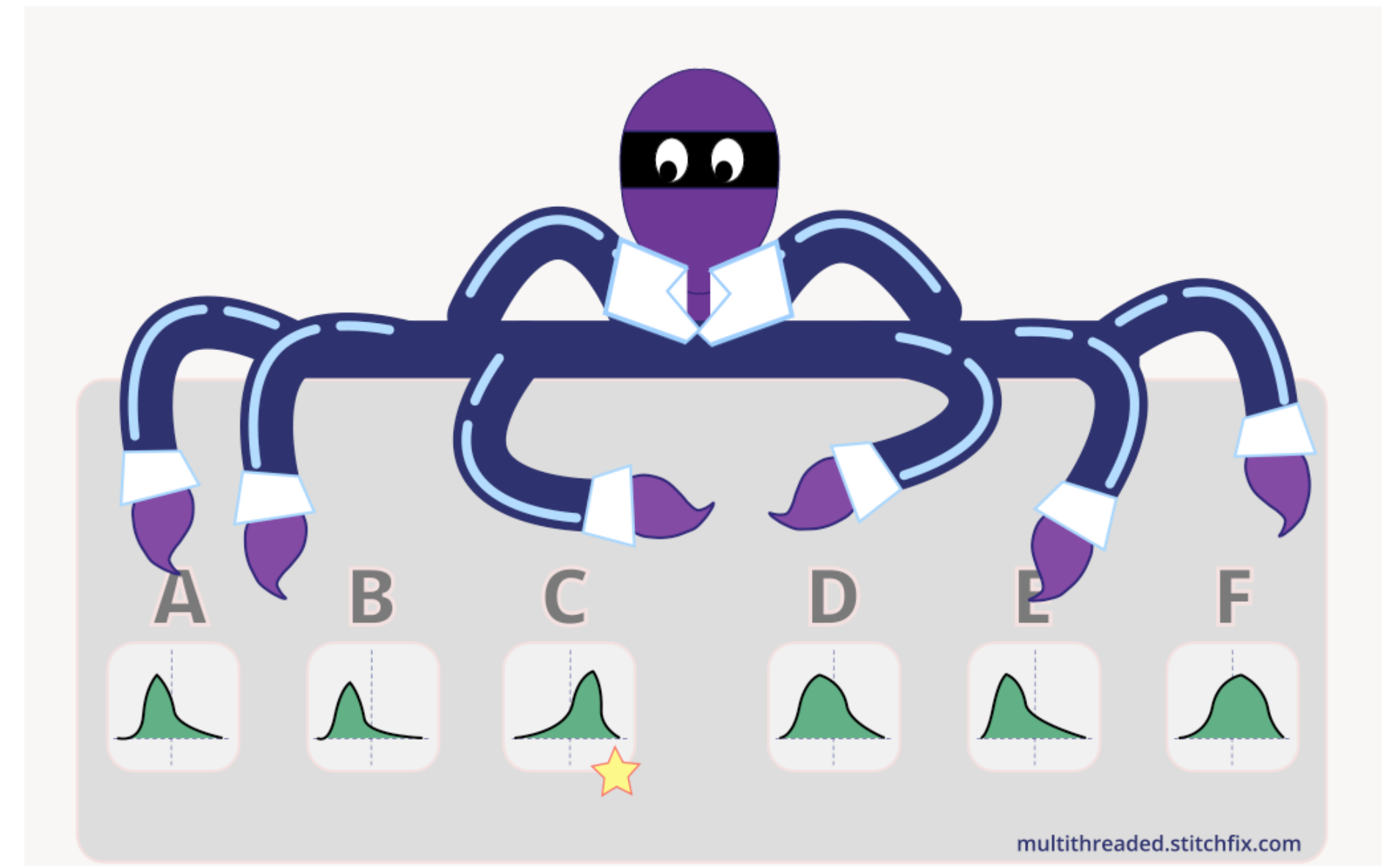
Multi-armed Bandit

- The initial state is fixed;
- The episode ends after the first step so we have a single state in the environment;
- An agent is facing repeatedly a choice among K different actions;
- We do not know the reward distribution for each of the K actions.

Multi-armed Bandit

- $\{p(r | a) | a \in \mathcal{A}\}$ is a set of reward's distributions;
- On each step, t an agent chooses a_t and get reward $r_t \sim p(\cdot | a_t)$

The agent's goal is to maximise $\mathbb{E}_{r \sim p(\cdot | a)}[\sum_{t=1}^T r_t]$ by choosing an action at each step.



Source

Examples

- News website: pick an article to show to a user to maximize the total number of clicks.
- Medical trials: pick treatment for a patient to cure as much people as possible.

For both examples $r \sim \text{Bernoulli}(p_a)$ with the unknown probability for each arm.

Thank you for your attention!